

On the Closure of Compositional Hierarchies in Digital Symmetries

A rigorous researcher-engineer bridging abstract mathematics and concrete digital systems.

June 13, 2026

Contents

1	Title Page(s)	2
2	Front Matter	1
2.1	Preface	1
2.2	Notation & Conventions	1
2.3	Methodological Protocol	3
3	Chapters	4
3.1	Part I: Foundations	4
3.1.1	1. Orientation & Thesis: Closure of Compositional Hierarchies in Digital Symmetries	4
3.1.2	2. Operads, PROPs, and Symmetric Monoidal Foundations	6
3.1.3	3. Closure & Clones: Post’s Lattice for (Bool)	7
3.2	Part II: Constructions and Modularity	10
3.2.1	4. Hierarchical Modularity & Contracts	10
3.2.2	5. Sequential Composition, Feedback, and Traces	12
3.2.3	6. Constructive Logic: Heyting Semantics for Digital Composition	13
3.2.4	7. Case Study—From NAND to XOR to Half-Adder	15
3.3	Part III: Symmetries and Applications	16
3.3.1	8. Symmetry Groups of Boolean Functions and Circuits	16
3.3.2	9. Reliability, Noise, and Robust Closure	18
3.3.3	10. Algebra of Programs & Hardware DSLs	21
3.3.4	11. Dataflow, Graphs, and Parallel Symmetries	23
3.3.5	12. Reversible & Quantum-Flavored Hierarchies	24
3.3.6	13. Cryptographic Constructions & Symmetry Constraints	25
3.3.7	14. Learning on $(\mathbb{B}^n)$ and Equivariant Architectures	27
3.3.8	15. Algorithms for Closure & Symmetry Analysis	29
3.3.9	16. Case Studies & Build Logs	30
4	Bibliography	33
5	Back Matter	34
5.1	Appendices	34
5.2	Glossary & Symbol Index	35
5.3	Artifacts & Reproducibility	38
5.4	Errata & Change Log	40

1 Title Page(s)

On the Closure of Compositional Hierarchies in Digital Symmetries

A technical study of compositionality, closure, and symmetry in digital systems

Author

This monograph develops a rigorous framework connecting category theory, Boolean function theory, and applications in hardware, software, quantum computation, cryptography, and machine learning. It emphasizes executable artifacts, reproducibility, and multi-resolution error metrics.

Draft • June 13, 2026

On the Closure of Compositional Hierarchies in Digital Symmetries

Front matter and technical monograph

CC-BY-4.0

Version 0.1.0

Created 2026-06-12

Maintained by author

Prepared for readers with a background in discrete mathematics, basic category theory,
and digital logic

2 Front Matter

2.1 Preface

Motivation. This book develops a compositional view of digital systems in which structure is not an afterthought but a source of mathematical leverage. The central themes are compositionality, closure, and symmetry: how complex systems are built from smaller ones, which families of operations remain stable under composition, and which invariances reveal hidden structure. These ideas recur across Boolean functions, circuits, categorical semantics, and related computational models, and they provide a common language for reasoning about hardware, software, and other discrete systems.

Audience and prerequisites. The intended reader is comfortable with discrete mathematics, basic category theory, and digital logic. Familiarity with sets, functions, relations, Boolean algebra, and elementary proof techniques will be useful throughout. Some chapters also assume a working knowledge of algebraic structures and diagrammatic reasoning, but the exposition is designed to build these ideas incrementally rather than presuppose advanced specialization.

How to use this book. The book is organized around a proof-as-builds workflow: mathematical claims are treated as constructions that can be checked, refined, and reused. Correspondingly, code is treated as witness material for the claims it implements, and each chapter is intended to leave behind concrete artifacts—definitions, derivations, examples, and verification traces—that can be inspected independently. Readers may proceed linearly, or use the chapter structure as a reference map: the early chapters establish the foundational language, the middle chapters develop modular constructions and feedback, and the later chapters apply the same principles to symmetry analysis, robustness, programs, dataflow, quantum-flavored models, cryptography, learning, and algorithms.

The guiding expectation is that each result should be understandable at multiple resolutions: as an abstract statement, as a diagrammatic or algebraic derivation, and as an artifact that can be reproduced or adapted. If you are interested primarily in applications, the later chapters can be read with selective consultation of the foundations; if you are interested in the theory, the book is designed to reward a careful sequential reading.

In short, this book aims to provide a unified framework for understanding how closure and symmetry organize digital composition, together with a disciplined method for turning that understanding into reusable technical artifacts.

2.2 Notation & Conventions

This chapter uses a compact, chapter-local notation policy: symbols are reintroduced at first use in each chapter when needed, so that definitions remain close to their point of application. When a symbol is reused with a specialized meaning in a later chapter, that meaning is stated explicitly in the local context.

Throughout the book, we write

$$\mathbb{B} := \{0, 1\}$$

for the Boolean domain, and

$$\mathbb{B}^n$$

for the set of bitstrings of length n . The notation

$$\text{Bool}(n)$$

denotes the set of all n -ary Boolean functions, that is, maps $f : \mathbb{B}^n \rightarrow \mathbb{B}$.

We use the following conventions for algebraic and compositional structure.

- $\mathcal{O}$ denotes an operad, and $\mathcal{O}(n)$ denotes the collection of operations of arity n .
- Σ_n denotes the symmetric group on n letters.
- $\circ$ denotes substitution or serial composition.
- $\otimes$ denotes parallel composition or monoidal product.
- Tr denotes feedback or trace.
- $\text{cl}(\cdot)$ denotes a closure operator.
- A *clone* is a set of operations closed under projections and composition.
- $G \curvearrowright X$ denotes a group action of G on X .
- $\text{Aut}(\cdot)$ denotes an automorphism group.

Diagrammatic and typing conventions are used consistently across the book. Ports are typed, and when a family of operations carries multiple sorts or colors, we treat it as a colored operadic structure. In such settings, the type of each port is part of the data of the diagram, and well-typed composition is assumed unless stated otherwise.

Error and resolution are measured by a chapter-specific quantity denoted $\varepsilon(r)$, where r indicates the relevant resolution or comparison regime. Unless a chapter overrides it, the default interpretation is normalized Hamming distance. Thus, for $f, g \in \text{Bool}(n)$, we write

$$\varepsilon_{\text{tt}}(r = n; f, g) := \frac{1}{2^n} \sum_{x \in \mathbb{B}^n} \mathbf{1}[f(x) \neq g(x)].$$

This quantity is the fraction of inputs on which f and g disagree. When a later chapter requires a different notion of discrepancy—for example, bisimulation distance, metastability-aware error, or another domain-specific metric—that chapter states the override explicitly and uses the same $\varepsilon(r)$ notation with the appropriate interpretation.

Unless otherwise indicated, equality is literal equality of the underlying mathematical objects, composition is read left-to-right in the order specified by the surrounding context, and all diagrams are assumed to be interpreted up to the equivalences declared in the relevant chapter.

For readability, we also adopt the following global conventions. First, subscripts and superscripts are used in the standard mathematical sense unless explicitly redefined. Second, when a chapter introduces a local abbreviation or overloaded symbol, the chapter should restate the intended meaning at the point of use. Third, any statement about equivalence, isomorphism, or invariance is to be read relative to the ambient structure currently under discussion (for example, set-theoretic, algebraic, categorical, or diagrammatic).

These conventions are intended to keep the exposition compact without sacrificing precision: local definitions control local meaning, while the book-wide defaults provide a stable baseline for Boolean, algebraic, and diagrammatic reasoning.

2.3 Methodological Protocol

This book is organized around a *claims-as-builds* protocol: each substantive claim is paired with an executable artifact, a test, or both. In practice, a theorem is not treated as complete until it is accompanied by a buildable representation of the statement, a reproducible derivation or proof sketch, and a checkable artifact that witnesses the intended behavior. This approach keeps the mathematical narrative and the implementation narrative aligned throughout the book.

The working protocol for each technical development is:

1. **Acquire** the relevant specifications, datasets, or formal statements.
2. **Calibrate** the error tolerance $\varepsilon(r)$ at the appropriate resolution or scale r .
3. **Log** the reproducibility metadata, including hashes, random seeds, toolchains, and any other build inputs needed to reconstruct the result.

The acquire step fixes the source material for a result: definitions, assumptions, input instances, and any external data or formal specifications. The calibrate step records the precision regime in which the claim is intended to hold, so that approximate statements, multi-resolution arguments, and robustness bounds can be compared on a common footing. The log step records enough provenance to make the result auditable and rerunnable, including versioned dependencies, compiler or interpreter settings, and the exact parameters used in experiments or checks.

Throughout the book, reproducibility is treated as part of the mathematical content rather than as an afterthought. When a chapter introduces a construction, the corresponding artifact should make clear what is being built, what is being tested, and what assumptions are required for the build to succeed. When a chapter proves an equivalence or closure property, the associated test should reflect the intended invariants at the relevant level of abstraction. In this sense, the protocol is not merely administrative: it is the mechanism by which claims are made inspectable.

The reading path is intentionally nonuniform. Readers with a stronger background in discrete mathematics or category theory may proceed directly into the foundational chapters, while readers who prefer a more concrete entry point may use the dependency structure as a guide and begin with the sections that emphasize examples, contracts, or case studies. The dependency graph across chapters is designed to support both styles: it identifies which results rely on earlier material and which chapters can be read with only the front matter and notation conventions in hand.

In particular, the book is arranged so that the early chapters establish the formal language, the middle chapters develop compositional constructions and proof-carrying artifacts, and the later chapters apply the same protocol to symmetry analysis, robustness, computation, and case studies. Readers are encouraged to treat the chapter dependencies as a map rather than a strict linear path: the map indicates where a result is used, what background it presupposes, and where a reader may safely branch depending on prior experience.

For practical use, the protocol can be summarized as follows. First, identify the claim and the artifact that should witness it. Second, determine the scale or tolerance at which the claim is meant to hold and record the corresponding $\varepsilon(r)$. Third, preserve the full build context so that the claim can be reconstructed later from the same inputs. This includes hashes for source files, seeds for randomized procedures, and the toolchain versions used to generate or verify the artifact. The result is a workflow in which proofs, tests, and build logs are all part of the same evidentiary chain.

The remainder of the book assumes this protocol implicitly. Claims should be read as buildable statements, constructions should be read as artifacts with interfaces and checks, and examples should be read as reproducible instances of the general theory. The chapter map should be used as

a dependency graph: follow it sequentially when you want the full development, or use it selectively when you want to enter through a concrete application and backfill the prerequisites as needed.

For orientation, the protocol also establishes a reading map across the book. The front matter supplies the notation and methodological commitments; the foundational chapters develop the abstract language of composition and closure; the middle chapters translate those ideas into modular constructions, feedback, and proof-carrying artifacts; and the later chapters apply the same discipline to symmetry, robustness, algorithms, and domain-specific case studies. Readers may therefore approach the book either top-down, by following the dependency graph chapter by chapter, or bottom-up, by starting from a concrete application and tracing the prerequisites backward until the relevant foundations are in place.

This reading map is meant to be practical rather than ceremonial. If a result depends on a prior definition, lemma, or convention, that dependency should be visible in the chapter structure and in the associated build artifacts. If a reader encounters a later chapter first, the dependency graph should indicate exactly which earlier material must be recovered before the claim can be checked or reused. In that sense, the book is not only a sequence of chapters but also a navigable system of dependencies, build steps, and reproducibility records.

3 Chapters

3.1 Part I: Foundations

3.1.1 1. Orientation & Thesis: Closure of Compositional Hierarchies in Digital Symmetries

This opening section fixes the vocabulary and scope for the rest of the book. The central claim is that many digital systems admit a common compositional description: once the relevant objects, symmetries, and composition laws are identified, one can ask when a class of constructions is *closed* under those laws and how such closure induces a *hierarchy* of increasingly expressive or increasingly constrained models.

We begin with the basic objects of study. In the Boolean setting, the ambient space is $\mathbb{B}^n$, the set of length- n bit vectors. From this starting point we will move among several closely related digital objects: Boolean functions $f : \mathbb{B}^n \rightarrow \mathbb{B}^m$, circuits, finite automata, and error-correcting codes. These are not treated as unrelated examples; rather, they are instances of structured finite systems whose behavior can be compared by composition, transformation, and equivalence. The book will repeatedly translate between a semantic view, where one studies the input–output behavior of an object, and a structural view, where one studies the wiring, state, or generating data that realizes it.

To compare such objects, we need a notion of error or distance that respects the level of description. At the level of a Boolean function, a natural metric is the truth-table Hamming distance, denoted $\varepsilon(r)$ when the object is viewed as a function of arity r . This measures disagreement between truth tables and is well suited to function-level approximation and classification. At the level of a circuit, the relevant notion is structural edit distance on the circuit graph: how many local changes are required to transform one wiring diagram into another. The two metrics serve different purposes, and the distinction matters throughout the book. Function-level proximity does not necessarily imply structural proximity, and vice versa. In later chapters, this separation will support multi-resolution comparisons between semantic equivalence, approximate equivalence, and implementation-level similarity.

Symmetry is the second organizing principle. A digital object may be invariant under a group

action, or it may transform equivariantly. In the Boolean setting, the most prominent symmetries arise from permutations of inputs, encoded by the symmetric group Σ_n , together with input and output negations. These generate the familiar NPN equivalence relation on Boolean functions: two functions are equivalent if one can be obtained from the other by permuting inputs, negating selected inputs, and optionally negating the output. Symmetry analysis will later be used both as a classification tool and as a design constraint. Invariance identifies features that remain unchanged under a symmetry action, while equivariance records the compatible transformation law of an output under transformed inputs.

Composition is the third pillar. The book uses several composition operations, each appropriate to a different level of abstraction. Sequential composition is written $\circ$, parallel or tensor composition is written $\otimes$, and feedback or trace is written Tr . These operations are not merely notation: they are expected to satisfy structural laws such as associativity, symmetry, and functoriality, so that complex systems can be assembled from smaller ones in a controlled way. The guiding idea is that a useful formalism should make composition explicit and should preserve the relevant equivalences under composition. This is the sense in which the book treats digital systems as compositional objects rather than as isolated artifacts.

The thesis of the book can now be stated informally. Digital systems often organize into compositional hierarchies: small components are combined into larger modules, modules into subsystems, and subsystems into architectures. A hierarchy is meaningful only when the composition laws are stable enough to support abstraction, and closure properties determine which operations remain inside a chosen class. In this sense, closure is not merely a set-theoretic property; it is a design principle that governs whether a family of digital objects can be manipulated internally without leaving the formalism. Hierarchy, correspondingly, is not just a ranking by size or complexity, but an organized stratification by expressive power, structural constraints, and closure strength.

We will use the term *closure* for the property that a class of objects is preserved under specified operations such as composition, tensoring, feedback, symmetry actions, or other admissible transformations. We will use the term *hierarchy* for an organized stratification of classes by expressive power, structural complexity, or closure strength. A hierarchy may be defined by increasing arity, by richer generators, by additional symmetries, or by stronger notions of equivalence. The point of the book is not that there is a single canonical hierarchy, but that many useful hierarchies can be studied with the same compositional language. This perspective will later connect Boolean clones, operadic generation, circuit families, and symmetry classes under a common framework.

A useful way to read the rest of the book is to keep three questions in view. First: what are the objects, and at what level are they being compared—semantic behavior, structural realization, or both? Second: what symmetries are being quotiented out, preserved, or exploited? Third: which compositions are allowed, and what closure properties do they induce? These questions recur in the foundational chapters on operads and PROPs, in the closure theory of Boolean functions, and in the later applications to circuits, automata, codes, and learning systems.

This orientation section therefore serves as a map. It introduces the objects $\mathbb{B}^n$, circuits, finite automata, and codes; the metrics $\varepsilon(r)$ and structural edit distance; the symmetry notions of invariance, equivariance, and NPN classes; and the composition laws $\circ$, $\otimes$, and Tr . Later chapters will make these ideas precise in the language of operads, PROPs, closure operators, and clone theory, but the conceptual target remains the same: to understand when digital composition is closed, how symmetry constrains that closure, and how hierarchies emerge from the interaction of both.

In the chapters that follow, these terms will be sharpened in progressively more formal settings. The foundational material will introduce the categorical and algebraic machinery needed to state closure theorems precisely, while the later parts of the book will show how the same vocabulary

applies across Boolean functions, circuits, automata, codes, and symmetry-aware learning systems. The present section is intentionally lightweight: it does not prove the main theorems, but it fixes the interpretation of the main nouns and verbs of the book so that the subsequent development can proceed without ambiguity.

3.1.2 2. Operads, PROPs, and Symmetric Monoidal Foundations

Foundational role. This section introduces the algebraic languages that organize compositional digital structure before one specializes to Boolean closure, circuit synthesis, or symmetry analysis. Operads capture substitutional composition of operations with multiple inputs and one output; PROPs extend this perspective to operations with many inputs and many outputs; and colored operads refine both by tracking port types. Together, these form a symmetric monoidal foundation for the later chapters on closure, modularity, and equivalence.

Operads: substitution, units, symmetric actions, and algebras. An operad packages families of operations $\mathcal{O}(n)$ for $n \geq 0$, together with a unit, symmetric group actions, and a substitution law. Informally, an element of $\mathcal{O}(n)$ is an n -ary operation, and operadic composition expresses the replacement of each input of an operation by another operation of matching arity. The unit represents the identity operation, while the symmetric action records invariance under permutation of inputs.

This structure is well suited to syntax: trees of operations can be composed by grafting, and the operad axioms ensure that substitution is associative and unital up to the specified symmetric bookkeeping. In applications, an *algebra* over an operad interprets abstract operations as concrete functions on a carrier object, thereby turning syntax into semantics. This passage from formal expressions to realized behavior is the first recurring theme of the book.

From syntax to semantics: free structures and quotienting. Operads support a standard two-step workflow. First, one forms a free operadic structure generated by a chosen set of primitive operations; this captures all formal composites built from the generators. Second, one imposes equations or identifications to quotient by observationally irrelevant distinctions. The result is a semantic object in which different syntactic presentations may denote the same behavior.

This free-then-quotient pattern will reappear throughout the book. It is the algebraic analogue of building a circuit language from primitive gates and then identifying circuits that compute the same function, or building a proof language and then quotienting by proof equivalence. The point is not merely to generate expressions, but to organize the space of expressions by the relations that matter for composition and interpretation.

PROPs: strict symmetric monoidal categories on $\mathbb{N}$. Where operads model one-output operations, PROPs model general wiring patterns with multiple inputs and multiple outputs. A PROP may be viewed as a strict symmetric monoidal category whose objects are natural numbers $0, 1, 2, \dots$, with monoidal product given by addition. Morphisms $m \rightarrow n$ represent processes or circuits with m input ports and n output ports.

This viewpoint is especially natural for digital systems. A circuit is not just a function; it is a wiring diagram with explicit interface arity, internal composition, and tensoring of independent components. Sequential composition corresponds to plugging outputs into inputs, while the monoidal product corresponds to placing components side by side. Symmetry morphisms encode wire permutations, so that the formalism can express reordering of ports without changing the underlying process.

In this book, PROPs provide the ambient language for circuits as morphisms and for wiring diagrams as compositional artifacts. They are the right setting for discussing equivalence of circuit presentations, structural rewrites, and the interaction between algebraic laws and graphical syntax.

Colored operads and port typing. Colored operads generalize ordinary operads by allowing operations to have typed inputs and outputs. Each color represents a port type, and an operation is only admissible when its input and output colors match the intended interface. This refinement is essential whenever a system has heterogeneous components, such as mixed signal types, data/control separation, or typed resource flows.

Colored operads make explicit what is often implicit in informal circuit diagrams: not every wire can connect to every port. The typing discipline constrains substitution and ensures that composition respects interface compatibility. In this sense, colored operads are a natural bridge between abstract operadic syntax and typed engineering practice.

Relation to Lawvere theories. Lawvere theories provide another algebraic presentation of finitary algebraic structure, emphasizing finite products and operations with multiple inputs but a single output. They are closely related to operads and can often be compared through the lens of arity, substitution, and equational presentation. For single-sorted algebraic theories, Lawvere theories and operadic formalisms frequently encode similar information, though with different categorical emphases.

For the purposes of this book, the key point is that these frameworks are complementary descriptions of compositional structure. Operads emphasize substitutional syntax; PROPs emphasize general wiring and symmetric monoidal composition; colored operads emphasize typed interfaces; and Lawvere theories emphasize algebraic operations governed by finite-product structure. The choice of formalism depends on whether one wants to foreground syntax, wiring, typing, or equational semantics.

Observational equivalence on PROP morphisms. To compare PROP morphisms in a way that is sensitive to intended semantics, we introduce an observational equivalence relation $\varepsilon(r)$, indexed by a resolution or regime parameter r . Two morphisms are $\varepsilon(r)$ -equivalent when they are indistinguishable at the chosen observational scale. In the default logical regime, denoted $r = \text{spec}$, $\varepsilon(\text{spec})$ is taken to mean ordinary logical equivalence: two morphisms are identified when they induce the same specified behavior.

This notation is a reminder that equivalence is not always absolute; it may depend on the granularity at which a system is observed, tested, or specified. In later chapters, this idea will support multi-resolution reasoning about circuits, programs, and symmetries. For now, it serves as the semantic counterpart to the syntactic identifications imposed by quotienting.

Summary. Operads, PROPs, and colored operads provide the categorical infrastructure for the rest of the book. Operads organize substitution; PROPs organize multi-input, multi-output composition; colored operads enforce typed interfaces; and Lawvere theories situate these ideas within algebraic semantics. The recurring pattern is that of free generation followed by quotienting under an equivalence such as $\varepsilon(r)$, with $\varepsilon(\text{spec})$ serving as the default notion of logical sameness.

3.1.3 3. Closure & Clones: Post’s Lattice for $\mathbf{Bool}$

This section develops the Boolean case of closure theory in a form that is both algebraic and algorithmic. The central objects are *Boolean operations* $f: \mathbb{B}^n \rightarrow \mathbb{B}$, where $\mathbb{B} = \{0, 1\}$, together

with the closure operators they generate under composition, permutation of variables, and insertion of projections. In the Boolean setting, these closure phenomena are organized by *clones*, and the global structure of all clones on $\mathbb{B}$ is described by Post's lattice.

A useful way to compare Boolean functions in this book is via a multi-resolution error metric. For two functions $f, g: \mathbb{B}^n \rightarrow \mathbb{B}$, one may measure disagreement by the Hamming distance of their truth tables, and then add a penalty when the functions do not use the same variable set or arity profile. We write this schematically as

$$\varepsilon(r)(f, g) = d_{\text{Ham}}(\text{tt}(f), \text{tt}(g)) + \lambda \pi(f, g),$$

where $\text{tt}(f)$ denotes the truth table of f , d_{Ham} is the Hamming distance on truth tables, $\pi(f, g)$ is a mismatch penalty for variable or arity differences, and $\lambda \geq 0$ is a weighting parameter. The exact choice of penalty is application-dependent, but the guiding principle is stable: semantic disagreement should be measured not only by output bits, but also by structural mismatch in the representation.

Closure operators and projections. A closure operator on a set of Boolean functions is a map C satisfying extensivity, monotonicity, and idempotence. In the present context, the closure of a set F of Boolean functions is the smallest class containing F that is closed under composition and under the insertion of projections. The projections are the coordinate maps

$$\pi_i^n(x_1, \dots, x_n) = x_i, \quad 1 \leq i \leq n,$$

and they play the role of the identity morphisms for function composition. Closure under projections ensures that variables may be selected, duplicated, or ignored as needed when building larger Boolean expressions from smaller ones.

Clones. A *clone* is a set of finitary operations on $\mathbb{B}$ that contains all projections and is closed under composition. Equivalently, a clone is a compositional universe of Boolean operations stable under substitution of operations into operations. Clones are the natural algebraic objects for describing what can be built from a given collection of gates or primitives. If F is a set of Boolean functions, then the clone generated by F , denoted $\langle F \rangle$, is the smallest clone containing F .

This perspective makes functional synthesis precise. Given a target Boolean function h , the question is whether $h \in \langle F \rangle$. If so, then h can be realized by a finite composition of functions from F and projections. If not, then F is insufficient for the desired expressive power, regardless of how deeply one composes its members.

Precomplete classes. A clone is called *precomplete* or *maximal proper* if it is a proper subclone of the full clone of all Boolean functions, but adding any function outside it yields the full clone. Precomplete classes mark the immediate boundary between incomplete and functionally complete systems. In hardware terms, they identify the last expressive classes before universality is obtained; in algebraic terms, they are the coatoms of the clone lattice.

Post's lattice. Post's lattice is the complete classification of all clones on $\mathbb{B}$. It organizes Boolean clones into a finite lattice of structurally significant families and their intersections. Among the most important named classes are the following:

- M : the monotone clone, consisting of functions preserving the pointwise order on $\mathbb{B}^n$;

- L : the affine clone, consisting of functions representable as affine polynomials over $\mathbb{F}_2$;
- D : the self-dual clone, consisting of functions satisfying $f(\bar{x}) = \overline{f(x)}$;
- T_0 : the clone of functions preserving 0;
- T_1 : the clone of functions preserving 1.

These classes are not isolated curiosities; they are the principal axes along which Boolean expressivity decomposes. For example, monotonicity constrains the direction of information flow, affine structure constrains parity behavior, and self-duality encodes a symmetry under bit-flip complementation.

The lattice viewpoint is especially useful because it turns expressive power into order-theoretic inclusion. If $C_1 \subseteq C_2$, then every function definable in C_1 is also definable in C_2 . Thus, moving upward in Post’s lattice corresponds to gaining expressive power, while moving downward corresponds to imposing stronger invariants.

Functional completeness. A set of Boolean functions is *functionally complete* if it generates the full clone of all Boolean functions on $\mathbb{B}$. Classical criteria due to Sheffer characterize when a single binary connective suffices. In particular, NAND and NOR are functionally complete: each alone generates all Boolean functions. More generally, a set of functions is complete precisely when it is not contained in any of the maximal proper clones identified by Post’s classification. This yields a decision procedure: to test completeness, it suffices to check whether the generating set lies inside one of the precomplete classes.

In practice, this criterion is often used in two directions. First, it certifies universality of a gate set. Second, it proves impossibility: if a candidate library is contained in M , for example, then it cannot express negation and therefore cannot be complete. Similar obstructions arise for L , D , T_0 , and T_1 .

Decision procedures and algorithms. The classification of Boolean clones supports concrete algorithms for synthesis and verification. A typical workflow is as follows:

1. normalize the candidate functions by truth table or algebraic normal form;
2. test membership in the principal Post classes M , L , D , T_0 , and T_1 ;
3. infer the smallest clone containing the set, or at least a tight upper bound on it;
4. determine whether the set escapes every maximal proper clone, hence is functionally complete.

These tests can be implemented using truth-table inspection, polynomial-time checks for the named classes, and search over canonical representatives of clone families.

A related computational problem is clone membership: given a target function h and a generating set F , decide whether $h \in \langle F \rangle$. For small arities, brute-force enumeration of the closure may be feasible; for larger instances, one typically relies on structural invariants, normal forms, or reductions to known clone classes. The book will later reuse these ideas in symmetry analysis and in algorithmic closure checking.

NPN classes and canonicalization. Boolean functions are often compared up to negation, permutation, and input negation, the so-called NPN transformations. These equivalence classes are useful for reducing redundancy in search and for identifying canonical representatives of gate libraries. While Post’s lattice classifies closure under composition, NPN classification organizes functions by symmetry. The two viewpoints complement each other: NPN equivalence simplifies the search space, while clone membership determines expressive closure.

Minimal generating sets. A generating set for a clone is minimal if no proper subset generates the same clone. Minimal generating sets are important both theoretically and practically. Theoretically, they expose the irredundant primitives underlying a class of Boolean behavior. Practically, they identify compact gate libraries and reduce synthesis complexity. In the Boolean setting, minimality is often studied together with canonical forms and with the precomplete classes of Post’s lattice.

The overarching message is that Boolean expressivity is not a flat collection of gates but a structured hierarchy of closure classes. Projections provide the base level of composition, clones capture substitutional closure, Post’s lattice classifies the resulting universe, and functional completeness identifies the boundary where a gate set becomes universal. This algebraic picture will serve as the foundation for later chapters on modular circuit construction, symmetry analysis, and algorithmic verification.

3.2 Part II: Constructions and Modularity

3.2.1 4. Hierarchical Modularity & Contracts

This section develops the modular layer of the book: how open circuits are packaged as reusable artifacts, how their boundaries are specified by interfaces and contracts, and how refinement supports hierarchical composition without losing correctness. The guiding idea is that a circuit is not merely a graph of gates, but an artifact with an exposed boundary, an internal implementation, and a measurable risk of contract violation at that boundary.

Let M denote a module with interface I , internal implementation P , and a contract C describing admissible boundary behavior. In the open-circuit setting, the interface records the arity, polarity, and typing of ports, while the contract constrains the observable relation between inputs and outputs. A module is reusable when any context that satisfies the interface assumptions can compose with the module without violating the contract. In practice, this means that the module must be parameterized by the data it depends on, rather than hard-coding assumptions that are only valid in one embedding. Parameterization is therefore not a convenience but a reusability condition: it isolates the degrees of freedom that may vary across contexts while keeping the boundary specification stable.

A useful quantitative summary of boundary reliability is the contract violation probability $\varepsilon(r)$, where r denotes the resolution level, abstraction scale, or refinement depth at which the module is inspected. At coarse resolution, $\varepsilon(r)$ measures the chance that a composed artifact fails to satisfy its declared boundary conditions when instantiated in a larger system. As refinement proceeds, one seeks a family of modules $\{M_r\}$ such that $\varepsilon(r)$ decreases, or at least remains controlled, under composition. This makes the boundary contract an executable specification rather than a purely informal promise. In the intended workflow, $\varepsilon(r)$ is not merely reported after the fact; it is tracked as part of the artifact’s build and test metadata so that regressions in boundary behavior are visible at the same level as type or interface failures.

The categorical language for open circuits is provided by hypergraph categories and decorated

cospan. In this view, an open circuit is represented by a cospan of interfaces into an internal wiring diagram, together with decoration data encoding the circuit structure. Hypergraph categories supply the algebraic operations needed to compose such open systems along shared boundaries, while preserving the ability to reason about ports, copying, and merging. Decorated cospans are especially well suited to modular hardware because they separate the external interface from the internal realization: two implementations with the same boundary behavior may be substituted without changing the surrounding composition. This separation is what makes the interface contract meaningful across different implementations, and it is also what allows a library of components to be reused in multiple higher-level assemblies.

This separation supports hierarchical modularity. Suppose a module M is refined into a sub-module decomposition $M_1, \dots, M_k$. A refinement map relates the abstract specification of M to the concrete implementation assembled from the M_i . The key requirement is that refinement preserve the contract: if the abstract module satisfies a property P , then the refined implementation should satisfy the corresponding instantiated property, possibly under additional assumptions made explicit in the interface. Hierarchical invariants are those properties that remain stable under repeated refinement and recomposition. Examples include arity preservation, type compatibility, monotonicity of a safety condition, and bounded error accumulation across levels. In a well-formed hierarchy, these invariants compose: if each refinement step preserves the relevant boundary data, then the entire tower of refinements remains contract-compatible with the original specification.

The compositional principle can be stated informally as follows: if each component module satisfies its local contract, and if the interfaces are matched correctly, then the assembled system satisfies the global contract up to the explicitly tracked boundary error. This is the mechanism by which local proofs scale to larger designs. It also clarifies the role of parameterization: a module should expose precisely the parameters that affect its contract, so that refinement can substitute implementations without changing the external proof obligations. When a refinement introduces new internal structure, the additional detail should be invisible at the boundary except insofar as it improves the estimate of $\varepsilon(r)$ or strengthens the available invariants.

A concise way to package these ideas is to treat a module as a triple (I, P, C) together with a refinement witness $M' \preceq M$ indicating that the concrete artifact M' implements the abstract contract of M . Composition then becomes a boundary-matching operation on interfaces, while refinement becomes a proof obligation that the induced boundary map preserves the contract. In this language, hierarchical modularity is not merely tree-shaped decomposition; it is a disciplined propagation of interface data, contracts, and witnesses through successive levels of abstraction.

From an engineering perspective, the artifact corresponding to this section is a contract-checked library with continuous integration tests. Such a library packages modules together with interface declarations, refinement witnesses, and executable checks that validate boundary conditions on representative inputs. The CI pipeline should verify at least three layers of correctness: syntactic well-formedness of the interface, semantic preservation of the declared contract under composition, and regression tests for the refinement maps that connect abstract and concrete versions of the same module. In a mature workflow, the library becomes a reusable repository of open-circuit components whose contracts are machine-checked and whose hierarchical invariants are exercised automatically. A practical implementation may also record the measured or estimated $\varepsilon(r)$ for each release, so that the build history documents not only whether the contract holds, but how robustly it holds across refinement levels.

A concrete example clarifies the intended use of these abstractions. Consider a parameterized adder block whose interface exposes bit-width n , carry-in, and carry-out ports, while its internal implementation may be a ripple-carry, carry-lookahead, or hybrid design. The contract specifies the arithmetic relation on the boundary, together with admissible timing or error bounds. A refinement

from a ripple-carry implementation to a carry-lookahead implementation changes the internal structure but should preserve the same external contract. If the library records the refinement witness and the CI suite checks the boundary relation for representative widths, then the artifact can be reused safely in larger arithmetic systems without re-proving the interface properties from scratch. This is the practical meaning of hierarchical modularity: the abstract specification remains stable while the implementation evolves beneath it.

The overall lesson is that modularity in open circuits is not only a matter of code organization. It is a mathematical discipline in which interfaces, contracts, and refinement maps jointly determine whether a component can be safely reused. Hypergraph categories and decorated cospans provide the compositional semantics; $\varepsilon(r)$ provides a boundary-sensitive error metric; and contract-checked artifacts provide the executable evidence that the hierarchy is stable under assembly and refinement. This section therefore sets up the later discussion of sequential composition and feedback by establishing the boundary discipline that makes larger compositions meaningful in the first place.

3.2.2 5. Sequential Composition, Feedback, and Traces

This section develops the transition from purely combinational composition to stateful and feedback-driven systems. The central theme is that once a circuit or machine can remember past inputs, the semantics of composition must account for time, recurrence, and fixed points. In this setting, sequential behavior is modeled by machines such as Mealy and Moore automata, by guarded recursive equations, and by categorical feedback operators captured abstractly by traced categories.

A Mealy machine produces outputs as a function of the current state and input, while a Moore machine associates outputs with states alone. Both can be viewed as finite-state realizations of a causal input–output map, and both admit composition laws that are richer than those of combinational gates. In particular, sequential composition must respect the propagation of state across time steps, so that the output at one stage may become the input to another stage in the next stage. This temporal dependency is the source of feedback, and feedback is the point at which algebraic and operational descriptions meet.

Guarded recursion provides one disciplined way to define feedback. Rather than allowing arbitrary self-reference, guarded equations require that recursive occurrences be delayed by at least one time step or otherwise protected by a causal constructor. This restriction ensures that the resulting system has a well-defined unfolding semantics and avoids ill-posed instantaneous loops. In categorical terms, traced monoidal structure abstracts the operation of feeding an output back into an input while preserving coherence laws for composition, tensoring, and iteration. The trace formalizes the idea that a feedback loop can be treated as a morphism-level operator, not merely as an ad hoc wiring pattern.

The hardware interpretation of feedback is especially important for latches and flip-flops. Standard storage elements can be built from cross-coupled NAND or NOR gates, and these constructions illustrate how sequential state emerges from combinational primitives plus feedback. Such elements also expose the distinction between synchronous and asynchronous closure. In synchronous designs, state changes are coordinated by a clock, and feedback is typically constrained to avoid race conditions and metastability. In asynchronous designs, closure under feedback is more delicate: the circuit must settle to a stable configuration without relying on a global timing discipline. The same structural loop may therefore be benign in one regime and hazardous in another.

Metastability provides a quantitative measure of this hazard. If $\varepsilon(r)$ denotes the probability of metastability as a function of a relevant parameter r such as resolution time, then the reliability of a feedback loop can be summarized by a mean time between failures (MTBF) estimate derived from $\varepsilon(r)$. In practice, the MTBF under feedback depends on the loop topology, the timing margins,

and the physical characteristics of the storage element. The formal role of $\varepsilon(r)$ in this section is to connect abstract feedback semantics with an operational reliability metric: a feedback law is not only well-defined or undefined, but also more or less robust under physical implementation.

Sequential composition also interacts with minimization and symmetry. Automata minimization identifies states that are behaviorally equivalent, collapsing a machine to a smaller representative without changing its input–output behavior. This process preserves the observable symmetries of the machine while removing redundant internal structure. From the perspective of closure, minimization is a canonicalization step: it selects a normal form within an equivalence class of sequential realizations. Such normal forms are useful both for reasoning and for implementation, because they support comparison, optimization, and reuse.

The symmetry viewpoint is especially relevant when feedback introduces multiple equivalent internal encodings of the same behavior. A minimized automaton may retain only the symmetries that are semantically meaningful, while discarding accidental symmetries introduced by a particular construction. This distinction matters for formal verification, where one often wants to prove that two implementations are equivalent up to state renaming, retiming, or other structure-preserving transformations. In this sense, sequential equivalence is not merely extensional equality of traces; it is often mediated by a chosen notion of preserved symmetry.

The section culminates in an executable artifact: an HDL implementation paired with formal equivalence checking, typically via an SMT-based backend. The workflow is to express the sequential design in hardware description language, generate a reference model or specification, and then discharge the equivalence obligation with a solver. This artifact makes the abstract discussion operational: Mealy/Moore semantics, guarded recursion, feedback, and minimization are all reflected in a machine-checkable build. When the equivalence check succeeds, it certifies that the feedback-rich implementation realizes the intended sequential behavior; when it fails, the counterexample often reveals a timing, state, or symmetry mismatch that can guide refinement.

Taken together, these ideas establish a bridge from algebraic composition to stateful systems. Sequential machines provide the operational model, traced categories provide the compositional calculus, guarded recursion provides a disciplined fixed-point principle, and formal equivalence checking provides the verification layer. The result is a framework in which feedback is not an exception to composition, but a structured and analyzable form of it.

For implementation-oriented readers, the section’s artifact can be summarized as follows: a sequential specification is encoded in HDL, a reference semantics is derived from the chosen machine model, and an SMT-backed equivalence check compares the implementation against the specification under the relevant clocking and reset assumptions. This makes the feedback loop explicit at the level of both design and proof, and it provides a concrete bridge from the categorical account of trace to the engineering practice of sequential verification.

3.2.3 6. Constructive Logic: Heyting Semantics for Digital Composition

Constructive logic provides the semantic discipline for digital composition when one wants proofs that can be executed, transformed, and checked alongside the circuits they justify. In contrast to classical reasoning over `Bool`, constructive reasoning distinguishes between the existence of a value and the availability of a witness. For circuit design, this distinction is not merely philosophical: it determines whether a specification can be turned into a proof-carrying artifact whose correctness is preserved by construction.

A useful way to organize this section is through the error profile $\varepsilon(r)$, interpreted as a tradeoff between proof obligations and counterexample rate. Here r may be read as a resource parameter, refinement level, or search budget, depending on the verification workflow. As proof obligations

increase, the counterexample rate should decrease; conversely, a low proof burden may leave more room for spurious implementations. The point of a constructive semantics is to make this tradeoff explicit and machine-checkable rather than implicit.

At the algebraic level, Heyting algebras generalize Boolean algebras by replacing excluded middle with an implication operation that is adjoint to conjunction. This matters for digital composition because many specifications are naturally intuitionistic: one can prove that a circuit satisfies a property without asserting that every proposition is decidable in advance. In a Boolean algebra, every element has a complement and every proposition is either true or false. In a Heyting algebra, negation is defined as implication into bottom, and double negation need not collapse to the original proposition. This asymmetry is precisely what makes constructive semantics sensitive to computational content.

The constructive XOR illustrates the difference. Classically, XOR is often treated as a derived Boolean connective with a simple truth table. Constructively, one must account for how the output is witnessed from the inputs. A constructive XOR specification can be expressed as a disjoint sum of cases, or as a proposition that carries enough information to extract a circuit without appealing to nonconstructive case splits. This is especially important when the target artifact is not just a truth-functional relation but an executable term.

The Curry–Howard correspondence supplies the bridge from logic to circuits: types as specifications, terms as implementations, and proofs as programs. Under this interpretation, a proposition about a digital component becomes a type, and a proof term inhabiting that type becomes a circuit or program satisfying the specification. The extraction pipeline then turns formal derivations into executable artifacts. In practice, this means that a proof in a dependently typed system can be compiled into a circuit description, preserving the logical structure of the specification in the generated implementation.

This perspective is particularly effective for closure questions. If a family of circuit specifications is closed under constructive connectives such as conjunction, disjunction, and implication, then one can build larger verified systems from smaller verified parts. Conjunction corresponds to pairing proofs and composing specifications in parallel; disjunction corresponds to tagged choice; implication corresponds to a transformation from one verified interface to another. These closure properties are the constructive analogue of algebraic closure under operations on Boolean functions, but now the operations carry proof terms as well as truth conditions.

However, double-negation caveats must be handled carefully. A statement of the form $\neg\neg P$ does not generally yield P constructively, so any workflow that relies on classical elimination principles must either justify them separately or restrict attention to fragments where they are admissible. For digital composition, this means that some seemingly natural equivalences are not available without additional assumptions, and the proof architecture must reflect that limitation. In particular, a proof that only establishes $\neg\neg$ of a specification is not yet a proof-carrying implementation.

A compact way to state the constructive discipline is to treat specifications as inhabited types and implementations as witnesses. Then the verification task is not merely to show that a relation holds, but to exhibit a term whose computational behavior realizes the relation. In a Heyting semantics, this aligns the algebra of propositions with the algebra of proof objects: conjunction corresponds to pairing, disjunction to tagged choice, and implication to function space. The resulting semantics is compositional in the strongest sense, because the proof structure itself mirrors the structure of the circuit or module being assembled.

The practical artifact associated with this section is a proof-carrying development in a system such as Agda or Coq, together with extracted circuits. In such a workflow, one encodes the specification of a gate, module, or composition law as a type; proves the desired properties constructively; and then extracts an implementation that is correct by construction relative to the proof. The re-

sulting artifact is not merely a theorem statement but a reproducible build object: the proof script, the extracted term, and the corresponding circuit representation can all be checked independently.

Taken together, Heyting semantics, Curry–Howard, and proof extraction provide a disciplined foundation for digital composition. They clarify which logical principles are computationally meaningful, which equivalences are safe to use, and how to move from abstract specifications to verified implementations without leaving the constructive setting. In the broader development of this book, this section supplies the logical counterpart to the modular and traced composition principles developed earlier: where those sections organize how components connect, constructive logic governs what it means for a connection to be justified by a witness that can be extracted and reused.

In summary, the constructive viewpoint replaces mere satisfiability with witness-bearing realizability. That shift is what makes the section’s central metric $\varepsilon(r)$ meaningful: the more structure one proves constructively, the smaller the gap between specification and executable artifact, and the lower the expected counterexample rate in downstream composition and testing.

3.2.4 7. Case Study—From NAND to XOR to Half-Adder

Gate-level error and hazard metric. At the gate level, we measure implementation quality with an error functional $\varepsilon(r)$ on a realization r . In this section, $\varepsilon(r)$ is interpreted not only as a logical mismatch rate against the intended truth table, but also as a timing-sensitive hazard indicator: transient glitches are counted when a circuit violates an admissible timing window and produces an unstable output during evaluation. This makes $\varepsilon(r)$ suitable for comparing alternative realizations that are logically equivalent but differ in depth, fanout, and susceptibility to hazards.

Deriving basic gates from NAND. The NAND gate is functionally complete, so the standard Boolean operations can be synthesized from it. Writing $x \uparrow y$ for NAND, one obtains

$$\neg a \equiv a \uparrow a, \quad a \wedge b \equiv (a \uparrow b) \uparrow (a \uparrow b), \quad a \vee b \equiv (a \uparrow a) \uparrow (b \uparrow b).$$

These identities exhibit the usual depth/size trade-off: a single primitive gate can generate the full Boolean basis, but the resulting implementations may require additional levels of composition. In a compositional setting, the relevant question is not merely whether a function is definable, but how the chosen realization affects structural complexity, delay, and robustness under the chosen $\varepsilon(r)$.

XOR from NAND and alternative forms. The exclusive-or operation is central to arithmetic circuits and can be expressed in the familiar sum-of-products form

$$a \oplus b \equiv (a \wedge \neg b) \vee (\neg a \wedge b).$$

This form makes the symmetry of XOR explicit: swapping a and b leaves the function unchanged. It also suggests a direct NAND synthesis by first constructing $\neg a$, $\neg b$, the two conjunctions, and then their disjunction. Alternative realizations are often preferable in practice. For example, one may use the identity

$$a \oplus b \equiv (a \vee b) \wedge \neg(a \wedge b),$$

or a purely NAND-based network with shared intermediate nodes. Different realizations are logically equivalent but may differ in gate count, depth, and hazard profile; the choice of implementation should therefore be evaluated against both functional correctness and the timing-window criterion encoded by $\varepsilon(r)$.

Half-adder specification and implementation. A half-adder takes two input bits $a, b \in \{0, 1\}$ and produces a sum bit s and a carry bit c . Its specification is

$$s = a \oplus b, \quad c = a \wedge b.$$

Thus the half-adder decomposes addition into a parity component and a carry component. Using the NAND-derived basis above, one can implement the half-adder entirely from NAND gates by reusing subexpressions for $\neg a$, $\neg b$, and $a \wedge b$, then assembling $a \oplus b$ from those intermediates. The resulting circuit is a concrete witness that a small universal primitive suffices to realize a standard arithmetic block.

Invariance under input swap. The half-adder specification is invariant under exchanging the two inputs:

$$(a, b) \mapsto (b, a).$$

Indeed, both $a \oplus b$ and $a \wedge b$ are commutative, so the output pair (s, c) is unchanged by swapping the inputs. This symmetry is not merely algebraic decoration; it is a useful correctness property for testing and for reasoning about implementation equivalence. Any candidate realization should preserve this invariance, and a failure to do so indicates either a wiring error or an unintended asymmetry introduced by the chosen decomposition.

Artifact: Verilog and testbench. The case study is intended to be accompanied by an executable artifact consisting of a Verilog implementation of the NAND-derived gates and half-adder, together with a testbench that enumerates the four input combinations $(a, b) \in \{0, 1\}^2$. The testbench should verify the truth table for s and c , and it should also check the swap invariance by comparing the outputs for (a, b) and (b, a) . In the presence of injected faults or timing perturbations, the same harness can be used to estimate $\varepsilon(r)$ empirically by counting mismatches and hazard events over the relevant input and timing scenarios.

Build-log perspective. From the standpoint of the book’s claims-as-builds protocol, this section is a small but representative build log. The specification begins with a Boolean relation, is refined into a NAND-only realization, and is then validated by executable tests. The important outputs are not only the final truth table, but also the intermediate identities, the structural choices that affect depth and sharing, and the invariance checks that expose accidental asymmetry. In this sense, the half-adder serves as a minimal benchmark for the broader methodology developed in the surrounding chapters.

Summary. Starting from NAND, one can derive negation, conjunction, disjunction, XOR, and finally a half-adder. The resulting chain of constructions illustrates the central theme of the book: closure under composition yields expressive power, while the choice of realization controls structural cost and robustness. The half-adder serves as a compact benchmark for both algebraic correctness and artifact-level testing under the error metric $\varepsilon(r)$.

3.3 Part III: Symmetries and Applications

3.3.1 8. Symmetry Groups of Boolean Functions and Circuits

This section studies symmetry as a structural invariant of Boolean functions and the circuits that realize them. The central objects are actions of permutation groups on input variables, induced

transformations on truth tables and circuit descriptions, and the resulting equivalence relations used to classify functions up to relabeling, negation, and output complementation. In this setting, symmetry is not merely an aesthetic property: it is an algorithmic handle for canonicalization, equivalence checking, and the extraction of compressed representations.

Let $f : \mathbb{B}^n \rightarrow \mathbb{B}$ be a Boolean function. The symmetric group Σ_n acts on $\mathbb{B}^n$ by permuting coordinates, and this action induces a corresponding action on functions by precomposition. A permutation $\pi \in \Sigma_n$ is an automorphism of f when

$$f(x_1, \dots, x_n) = f(x_{\pi(1)}, \dots, x_{\pi(n)})$$

for all inputs $x \in \mathbb{B}^n$. The set of all such permutations forms the automorphism group $\text{Aut}(f) \leq \Sigma_n$. More generally, the stabilizer of a function under a chosen group action records the symmetries that preserve the function exactly, while orbits partition the space of functions into equivalence classes under that action. These orbit–stabilizer notions are the basic language for symmetry-aware classification.

For circuits, the same viewpoint applies at the level of wiring diagrams and gate labels. A circuit automorphism may permute input pins, internal wires, or structurally identical subcomponents while preserving the computed function or the circuit graph up to isomorphism. This distinction matters: two circuits can compute the same Boolean function while having different internal symmetries, and conversely a circuit may exhibit graph-level symmetry that is not visible in the truth table alone. The book therefore treats function symmetry and circuit symmetry as related but not identical layers of structure. In practice, circuit-level symmetry information can be used to prune search, identify repeated substructures, and support canonical labeling of netlists.

A particularly important equivalence relation is NPN equivalence. Two Boolean functions f and g are NPN-equivalent if one can be obtained from the other by a combination of input negations, input permutations, and output negation. Concretely, there exist a permutation $\pi \in \Sigma_n$, a vector $a \in \mathbb{B}^n$ specifying input complements, and a bit $b \in \mathbb{B}$ specifying output complement such that

$$g(x) = f(x_{\pi(1)} \oplus a_1, \dots, x_{\pi(n)} \oplus a_n) \oplus b.$$

NPN equivalence is a standard classification notion in logic synthesis because it identifies functions that differ only by renaming and polarity changes of variables. Canonical forms for NPN classes provide a representative for each orbit, enabling database indexing, pattern matching, and symmetry-aware optimization. In practice, a canonicalizer must balance exactness, speed, and robustness on large truth tables or circuit instances. A useful canonicalizer therefore combines structural invariants, orbit pruning, and tie-breaking rules that are stable under repeated application.

The spectral perspective supplies a complementary set of tools. Writing the Walsh–Hadamard transform of f as $\widehat{f}(S)$ for subsets $S \subseteq \{1, \dots, n\}$, one obtains a frequency-domain representation that is sensitive to variable interactions and often easier to compare under symmetry. Approximate symmetry can be quantified by a spectral correlation score $\varepsilon(r)$, where r denotes a candidate relabeling or transformation and the score measures how closely the transformed spectrum matches the original. In this book, $\varepsilon(r)$ serves as a practical heuristic for detecting near-symmetries, ranking candidate automorphisms, and guiding search before exact verification. Spectral methods are especially useful when exact group computation is expensive, or when the object under study is noisy, incomplete, or only approximately symmetric. They also provide a natural bridge between exact symmetry and approximate invariance: a high correlation score suggests a plausible symmetry, while a low score can eliminate candidates early.

Beyond permutation symmetries, linear and affine transformations provide a broader algebraic framework. The general linear group $\text{GL}(n, 2)$ acts on $\mathbb{B}^n$ by invertible linear maps over $\mathbb{F}_2$, and the affine group $\text{AGL}(n, 2)$ extends this action by translations. These actions are relevant when one studies functions up to linear or affine equivalence, a coarser but often computationally tractable relation. Spectral methods interact naturally with these transformations because linear changes of variables reorganize Fourier coefficients in structured ways. This makes the Walsh–Hadamard domain a useful arena for both exact invariants and approximate matching under affine symmetries. In particular, spectral signatures can be used to cluster functions before exact orbit computation, reducing the cost of downstream canonicalization.

A practical symmetry-analysis workflow has two stages. First, a symmetry oracle proposes candidate automorphisms, stabilizers, or equivalence witnesses using structural cues, orbit information, and spectral signatures. Second, a canonicalizer normalizes the function or circuit to a chosen representative, producing a reproducible form suitable for comparison across instances. The oracle is intentionally permissive: it may return false positives, but it should be fast and high-recall. The canonicalizer is intentionally strict: it must verify the chosen representative and ensure that repeated application is idempotent. Benchmarks are used to evaluate both correctness and performance, with particular attention to the tradeoff between exhaustive group search and heuristic pruning. A practical benchmark suite should report not only runtime, but also orbit sizes discovered, canonical-form stability, and the rate at which spectral candidates are confirmed by exact checks.

For implementation, it is useful to separate three levels of output. At the first level, the system records invariants such as support size, algebraic degree, and coarse spectral statistics. At the second level, it enumerates candidate symmetries and computes stabilizer information for the most promising candidates. At the third level, it emits a canonical label or normal form together with a proof trace or witness certificate when available. This stratification makes the artifact easier to test and easier to integrate into synthesis or verification pipelines. It also clarifies where approximation is allowed: the oracle may be heuristic, but the canonicalizer and any reported automorphism should be exact.

The main conceptual thread is that symmetry is a closure phenomenon: once a function or circuit is placed under a group action, its meaningful invariants are those that survive the action, and its classification is governed by orbit structure. Automorphisms, stabilizers, and canonical forms provide the exact algebraic language; NPN equivalence provides the synthesis-oriented classification; and spectral methods provide scalable approximations and diagnostics. Together, these tools prepare the ground for later chapters on algorithmic canonicalization, equivalence checking, and symmetry-aware design. They also establish a reusable pattern for the rest of the book: identify the acting group, compute or approximate its orbits, and extract canonical representatives that support comparison, reuse, and verification.

3.3.2 9. Reliability, Noise, and Robust Closure

This section addresses the practical question of how compositional digital systems behave when gates, wires, or state updates are no longer ideal. In the noiseless setting, closure is usually discussed as an algebraic property: a class of functions is stable under composition, substitution, or wiring. Under noise, one asks for a stronger notion of *robust closure*: not only should the intended computation remain in the class, but its implementation should remain reliable under perturbation, faults, and finite redundancy.

A convenient summary statistic is the reliability function

$$\varepsilon(r) := 1 - \Pr[\text{error}],$$

as a function of a resource parameter r , such as depth, size, redundancy level, code distance, or number of correction rounds. In an idealized analysis, $\varepsilon(r)$ should improve with r for a well-designed fault-tolerant construction; in practice, it often exhibits a tradeoff between overhead and residual error. The central design problem is therefore to choose a composition scheme whose reliability improves faster than the noise accumulates.

Noisy gates, majority logic, and redundancy. A basic model assumes that each gate independently fails with some small probability p . Even when the underlying logical operation is simple, repeated composition can amplify error unless the architecture includes redundancy. The classical response is to replace a single logical operation by a gadget that computes the same function on multiple copies and then reconciles the outputs by a robust decision rule. Majority logic is the canonical example: if three copies of a bit are available and at most one is corrupted, the majority vote recovers the intended value. More generally, majority-based schemes implement a compositional symmetry: the logical value is invariant under permutations of the replicated wires, and the decision rule factors through this symmetry.

This symmetry perspective is useful because it explains why redundancy is not merely duplication. The replicated physical states are organized into equivalence classes, and the correction rule is designed to be insensitive to the admissible perturbations within each class. In that sense, majority logic is a small but concrete instance of robust closure: the logical operation survives because the implementation has been quotiented by a noise-tolerant symmetry.

Von Neumann schemes. Von Neumann’s approach to reliable computation formalizes this idea at scale. Instead of trusting a single noisy gate, one builds a circuit from noisy components together with redundancy and periodic correction. The key insight is that reliability can be made to improve recursively if the noise rate is below a threshold and if the redundancy overhead is controlled. In such schemes, the architecture itself becomes part of the computation: the logical function is not merely evaluated, but evaluated through a fault-tolerant encoding and decoding process.

A useful way to read these constructions is as iterative refinement. Each layer takes a noisy physical realization, extracts a more stable logical state, and then re-encodes it into a form that is again suitable for further computation. The resulting design is compositional in the strong sense that the correction mechanism composes with the computation rather than sitting outside it.

Error-correcting codes as compositional symmetries. Error-correcting codes provide a particularly clean language for robust closure. An encoding map embeds logical data into a larger physical space, while a decoding map projects back after noise. The code space is characterized by invariances and syndromes: many distinct physical states represent the same logical state, and the correction procedure identifies which perturbations are admissible. From the perspective of compositional algebra, a code acts like a symmetry-respecting section of a larger system. The logical operations are those that preserve the code space up to correctable error, so the code defines a closure condition relative to the noise model.

This viewpoint is useful because it separates three layers:

- the logical specification of the computation,
- the physical implementation subject to noise,
- the correction and decoding layer that restores the logical semantics.

A robust design is one in which these layers compose cleanly, so that the error introduced by one stage remains within the correction capability of the next. In practice, this means that the code distance, the gadget design, and the scheduling of correction rounds must be chosen together rather than independently.

Threshold theorems for reliable computation. Threshold theorems formalize the possibility of scalable fault tolerance. Informally, they state that if the physical error rate is below a certain threshold, then arbitrarily long computations can be performed reliably with only polylogarithmic or polynomial overhead, depending on the model and assumptions. The theorem is not a claim that noise disappears; rather, it asserts that the logical error can be driven down faster than the circuit grows, provided the architecture is sufficiently redundant and the noise is sufficiently weak.

In the language of this book, a threshold theorem is a closure result under perturbation: the class of computable functions remains stable when the implementation is lifted from ideal gates to noisy gadgets, as long as the noise lies in a regime where the correction scheme closes the error dynamics. The threshold is therefore a boundary between two qualitative behaviors: below it, recursive correction dominates; above it, error accumulation overwhelms the computation.

A schematic way to express this is to view the logical error rate after one protected layer as a function $f(p)$ of the physical error rate p . A threshold regime is one in which $f(p) < p$ for sufficiently small p , so repeated concatenation or repeated correction drives the effective error downward. The exact form of f depends on the code family and noise model, but the qualitative fixed-point picture is common across many fault-tolerant constructions.

Reliability as a design metric. The function $\varepsilon(r)$ is useful not only as a theoretical abstraction but also as an engineering metric. For a given family of implementations, one can measure how reliability changes with redundancy, depth, or code distance. Typical plots compare error probability against overhead, showing whether additional resources buy meaningful robustness. Such plots are especially informative when different fault-tolerance strategies are compared on the same benchmark.

In a well-behaved family, increasing redundancy should improve reliability up to the point where the overhead itself becomes the dominant cost. This makes $\varepsilon(r)$ a multi-objective quantity: one wants high reliability, but also acceptable size, latency, and energy use. The point of robust closure is not to eliminate tradeoffs, but to ensure that the tradeoffs remain controlled as the system scales.

Artifact: fault-injection harness. A practical way to study these effects is to build a fault-injection harness that perturbs gates, wires, or state transitions according to a chosen noise model and then measures the resulting output error rate. The harness can sweep over redundancy parameters and produce error-versus-redundancy plots, making the tradeoff between cost and reliability explicit. Such an artifact is valuable for validating analytic claims, for comparing majority-based and code-based schemes, and for identifying regimes where a proposed construction fails to achieve robust closure.

A useful harness reports not only the final error probability but also intermediate statistics such as syndrome rates, correction success rates, and the distribution of failure modes. Those measurements help distinguish between designs that fail because the noise model is too strong and designs that fail because the correction logic is structurally mismatched to the computation.

Summary. Reliability under noise is not an afterthought to compositional design; it is a structural constraint on what counts as a stable implementation. Majority logic, von Neumann-style

redundancy, and error-correcting codes all realize the same broad principle: replicate, detect, and correct so that the logical computation survives physical perturbation. Threshold theorems then provide the asymptotic guarantee that this principle can scale. In that sense, robust closure is the noisy analogue of algebraic closure: the system remains within its intended computational class even after the environment has acted on it.

3.3.3 10. Algebra of Programs & Hardware DSLs

This section develops the algebraic viewpoint that underlies program semantics and hardware domain-specific languages (DSLs): programs are not merely executed, but also interpreted as morphisms in an algebraic theory, transformed by equations, and validated by rewriting and property-based testing.

A useful starting point is the idea of a law coverage metric, written here as $\varepsilon(r)$, which measures how well a test suite or mutation campaign exercises the intended equational laws of a representation r . In the present setting, $\varepsilon(r)$ can be read as a property-coverage or mutation-score proxy for laws: if a DSL is equipped with rewrite rules, normal forms, or algebraic identities, then a high value of $\varepsilon(r)$ indicates that the generated tests are sensitive to violations of those laws. This is especially relevant for compiler passes and circuit optimizers, where a transformation may preserve extensional behavior while still breaking a stated equational invariant. The metric is therefore not a replacement for proof, but a practical complement to it: it helps quantify whether the laws that define the DSL are actually being exercised by the artifact.

The semantic backbone of many such DSLs is provided by Lawvere theories and their effectful extensions. A Lawvere theory presents an algebraic theory by specifying operations of finite arity together with equations, and it is well suited to describing compositional program fragments whose behavior is determined by generators and relations. When effects are present—for example, state, nondeterminism, exceptions, or resource usage—the pure equational picture must be enriched. In categorical terms, one often passes from ordinary algebraic theories to structures that account for effectful composition, while retaining the same guiding principle: syntax is generated freely, and semantics is obtained by quotienting by equations and interpreting in a model. This perspective connects naturally to operads and PROPs. Operads organize multi-input, single-output composition, while PROPs generalize to multiple inputs and multiple outputs, making them especially apt for circuits and signal-flow systems. Colored operads further refine the picture by tracking typed wires or heterogeneous interfaces. The common theme is that the algebraic presentation of a DSL should match the shape of the compositions it is meant to express.

For hardware DSL design, this algebraic viewpoint is not merely aesthetic; it is operational. A well-designed embedded DSL (EDSL) for circuits typically exposes a small set of combinators for primitive gates, wiring, duplication, deletion, and composition. The combinators should be chosen so that the resulting terms admit a useful normal form. Normal forms serve several purposes at once: they provide a canonical representation for equality checking, they support optimization by rewriting, and they make it possible to state and prove invariants about generated hardware. In practice, one often designs the DSL so that syntactic composition mirrors the intended circuit topology, while a separate normalization phase eliminates administrative overhead such as redundant identities, trivial permutations, or duplicated structure. The resulting artifact is easier to reason about because the algebra of the syntax aligns with the algebra of the semantics.

Rewriting systems supply the mechanism by which these normal forms are obtained. A rewrite rule is an oriented equation, and a collection of such rules induces a reduction relation on terms. Two classical properties govern the quality of a rewriting system. Termination ensures that repeated rewriting cannot continue indefinitely; confluence ensures that whenever a term can be rewritten in

more than one way, all reduction paths can be joined to a common result. When both properties hold, every term has a unique normal form, and equational reasoning becomes algorithmic: two terms are equal modulo the theory precisely when their normal forms coincide. In the context of program and hardware DSLs, this is a powerful design target. It allows one to encode algebraic identities as rewrite rules, prove that the rules are sound with respect to the intended semantics, and then use normalization as a decision procedure for a large class of equivalences.

Completeness is the bridge between the abstract equational theory and the concrete rewrite system. An equational presentation is complete when the rewrite rules are sufficient to derive all valid equalities of the theory, typically modulo the chosen notion of equivalence. In a DSL for circuits, completeness means that the optimizer or normalizer is not merely performing ad hoc simplifications, but is systematically capturing the intended algebra. This is particularly important when the DSL is used as a compilation target or as a proof language for hardware transformations. If the rewrite system is incomplete, then some semantically valid transformations remain inaccessible to the toolchain; if it is unsound, then the toolchain may silently change meaning. The ideal is therefore a rewrite system that is both sound and complete for the fragment of the theory that the DSL is meant to represent, with termination and confluence providing the computational discipline needed for reliable automation.

An artifact-oriented implementation of these ideas can be organized around a small DSL with equational reasoning and QuickCheck-style randomized testing. The DSL exposes constructors for the primitive operations of the domain, together with an interpreter into a semantic model and a normalizer based on rewrite rules. Equational laws are then encoded as properties: for example, that normalization preserves denotation, that a rewrite step is semantics-preserving, or that two syntactically distinct expressions are observationally equivalent. QuickCheck-style generators can produce random well-typed terms, including terms near the boundaries of the rewrite system where bugs are most likely to appear. The property suite can then be scored using $\varepsilon(r)$, giving a quantitative estimate of how thoroughly the laws are being exercised. In a mature workflow, the same laws appear in three places: as rewrite rules, as proof obligations, and as executable properties. Agreement among these three views is a strong indicator that the DSL is well founded.

The broader methodological lesson is that algebraic semantics, rewriting, and testing are not separate concerns but mutually reinforcing layers of the same design. Lawvere-theoretic syntax provides the generators and equations; operads and PROPs organize the shape of composition; rewriting turns equations into executable normalization; and property-based testing measures whether the intended laws are actually enforced by the implementation. For hardware DSLs in particular, this alignment is crucial because the cost of a semantic mismatch is not merely a failed test but a potentially incorrect circuit. The section therefore treats algebraic structure as an engineering resource: it constrains the language, enables optimization, and supports verification.

Artifact sketch. A representative deliverable for this section is a small circuit DSL with the following components: a typed syntax of combinators; a denotational interpreter into Boolean or signal-flow semantics; a rewrite-based normalizer; a library of equational laws stated as executable properties; and a randomized test harness that reports law coverage via $\varepsilon(r)$. Such an artifact makes the abstract discussion concrete and provides a reusable template for later sections on dataflow, reversible computation, and symmetry-constrained design.

Design note. Relative to the surrounding chapters, this section serves as the bridge from symmetry and closure theory to executable language design. It intentionally emphasizes the operational role of equations: in the earlier material, algebraic structure classifies functions and circuits; here,

the same structure becomes a programming discipline for building, normalizing, and testing DSLs. That bridge prepares the reader for later sections on dataflow graphs, reversible and quantum-flavored hierarchies, and algorithmic closure analysis.

3.3.4 11. Dataflow, Graphs, and Parallel Symmetries

This section studies how streaming computation, graph structure, and symmetry interact when systems are executed under resource constraints. The central theme is that a dataflow network is not only a wiring diagram for computation, but also an object whose symmetries can be exploited for scheduling, load balancing, and architecture-aware transformation.

A useful performance summary for a streaming system is the error functional $\varepsilon(r)$, where r denotes a resource regime or runtime configuration. In this setting, $\varepsilon(r)$ may be read as a multi-objective measure combining throughput, latency, and deadline miss rate. High throughput alone is not sufficient: a configuration is acceptable only when latency remains bounded and deadline misses stay within the target envelope. Thus, the operational question is not merely whether a pipeline computes the right function, but whether it computes it at the right rate and with the right temporal guarantees.

Kahn process networks provide a canonical model for deterministic streaming computation. Their compositionality makes them a natural fit for the book’s broader emphasis on closure: components can be connected by channels, and the resulting network can often be reasoned about by local properties of its parts. In the same spirit, streaming operads offer an algebraic language for composing stream processors with explicit interface arities. This viewpoint supports compositional scheduling, where the schedule of a large system is derived from the schedules of its subcomponents rather than imposed monolithically. The benefit is twofold: the semantics remain modular, and the implementation can be adapted to heterogeneous execution targets.

Graph structure becomes especially important when the dataflow graph admits nontrivial automorphisms. An automorphism of a pipeline preserves the connectivity pattern while permuting internal components, and such symmetries can be used to identify equivalent execution choices. In practice, graph automorphisms can expose load-balanced symmetries: if several subgraphs are structurally interchangeable, then work may be distributed across them in a way that preserves semantics while improving utilization. This is particularly relevant in replicated stages, fork-join patterns, and tiled computations, where symmetry can be turned into a scheduling resource.

The same structural ideas motivate fusion and fission laws. Fusion combines adjacent stages to reduce communication overhead and improve locality; fission splits a stage into parallel copies to increase throughput or match available hardware parallelism. These transformations are not purely syntactic optimizations: they must respect dependencies, buffering constraints, and the symmetry class of the underlying graph. When applied carefully, fusion and fission can be viewed as symmetry-preserving rewrites that move computation toward a more efficient realization without changing the observable stream behavior.

Hardware mapping makes these issues concrete. On GPUs and FPGAs, tiling is often the decisive step in obtaining good performance. A tile is a finite block of computation and data movement chosen to fit memory hierarchies, vector widths, or spatial fabric constraints. In a symmetric dataflow graph, tilings can be aligned with automorphism orbits so that repeated substructures are mapped uniformly across processing elements. This can reduce control complexity, improve cache or on-chip memory reuse, and make the implementation more predictable under scaling.

A useful way to organize the implementation is to treat the streaming kernel as a parameterized family indexed by symmetry transforms. One begins with an abstract stream specification, analyzes the graph-level symmetries, and then generates target-specific kernels for a chosen tiling or load-

balancing strategy. In this workflow, the symmetry analysis is not an afterthought: it guides the choice of decomposition, the placement of buffers, and the granularity at which parallelism is exposed. The resulting build log should record the chosen $\varepsilon(r)$ metrics, the detected graph automorphisms, and the effect of fusion/fission passes on throughput and deadline miss rate.

Taken together, these ideas show that streaming systems are best understood as compositional graphs with exploitable symmetry. The algebraic perspective clarifies when schedules can be derived locally, while the graph-theoretic perspective identifies equivalence classes of execution strategies. The practical outcome is a disciplined route from abstract dataflow specification to parallel implementation on modern hardware, with performance measured against explicit latency, throughput, and deadline constraints.

3.3.5 12. Reversible & Quantum-Flavored Hierarchies

This section develops a bridge from reversible classical computation to quantum-flavored compositional hierarchies. The common theme is that information-preserving structure constrains the available operations, while the chosen notion of error or approximation determines how far one may relax exact closure. We use the notation $\varepsilon(r)$ for a reversible cost measure, and we also refer to quantum error in the diamond norm when discussing approximate quantum implementations.

A reversible computation is one in which every step is bijective on the underlying state space. In the Boolean setting, this means that gates act as permutations on $\mathbb{B}^n$, so the corresponding circuit semantics live inside a group-like fragment of the usual circuit monoid. This restriction is not merely aesthetic: it encodes conservation of information and forces the design of circuits to respect global invertibility. As a result, reversible synthesis is naturally phrased as a constrained factorization problem inside a hierarchy of gate sets.

Canonical reversible primitives include the Toffoli gate and the Fredkin gate. The Toffoli gate is universal for reversible classical computation when ancillary bits are available, while the Fredkin gate provides a controlled swap that is especially useful for routing and conservative computation. These gates generate rich reversible clones, but the synthesis problem is rarely just about generation; it is also about managing ancillas and garbage. Bennett cleanup addresses this issue by computing a function reversibly, copying out the desired output, and then uncomputing the intermediate workspace so that the auxiliary state is restored. In categorical terms, this is a disciplined use of forward computation followed by a reverse pass that witnesses exact cancellation.

The reversible fragment interacts cleanly with linear and affine structure over $\mathbb{F}_2$. Linear reversible maps correspond to invertible matrices over $\mathbb{F}_2$, while affine maps add translations by constants. These classes capture parity symmetries and other invariants that are invisible to purely local gate descriptions. In particular, parity-preserving or parity-sensitive constraints can be used to classify which reversible transformations lie in a given hierarchy and which require genuinely nonlinear resources such as Toffoli-like control. This viewpoint is useful both for synthesis and for lower bounds: if a target transformation violates a conserved parity invariant, then it cannot be realized within a smaller affine fragment.

The reversible cost $\varepsilon(r)$ is best understood as a multi-resolution measure of how expensive it is to realize a reversible specification r under a chosen gate basis, ancilla budget, and cleanup policy. Depending on the application, this cost may count gate depth, gate count, ancilla usage, or a weighted combination of these quantities. The point of introducing such a measure is to compare exact reversible implementations with approximate or resource-relaxed ones in a principled way. In the classical reversible setting, exactness is usually the default; in the quantum setting, approximation becomes unavoidable, and the error metric must be explicit.

Quantum computation extends these ideas by replacing deterministic reversible maps with

unitary evolution on Hilbert spaces. Here the relevant notion of error is often the diamond norm, which measures the worst-case distinguishability of channels even when the system is entangled with an arbitrary reference. This makes the diamond norm a robust choice for compositional reasoning: it behaves well under tensoring with identities and under circuit composition, so it supports hierarchical error accounting. When a quantum synthesis procedure approximates a target unitary or channel, the diamond norm provides a stable way to state the approximation guarantee.

Within the standard gate model, Clifford+ T is a central universal set. Clifford gates alone admit efficient classical simulation and generate a highly structured subgroup of the unitary group, but they are not universal for arbitrary quantum computation. Adding the T gate yields universality, and the resulting hierarchy is a canonical example of a resource-sensitive quantum closure: Clifford structure supplies algebraic regularity, while T injects the non-Clifford power needed for full expressiveness. This mirrors the reversible classical story, where a small structured fragment becomes universal only after a carefully chosen nonlinear extension.

The ZX-calculus provides a diagrammatic language for reasoning about quantum processes. As a PROP, it organizes objects by wire count and morphisms by string diagrams, with composition and tensor product corresponding to sequential and parallel composition. This makes ZX-calculus especially well suited to the book’s compositional perspective: equations between diagrams become rewrite rules, and proof search becomes a controlled form of normalization. The PROP structure also clarifies how local rewrite principles scale to global circuit transformations, since the same generators and relations apply uniformly across arities.

From the standpoint of artifacts, two workflows are especially important. First, reversible synthesis can be implemented as a search-and-cleanup pipeline that produces exact reversible circuits together with explicit ancilla management. Second, ZX rewrites can be mechanized via a prover that checks diagrammatic equivalences and applies normalization strategies. In both cases, the artifact is not just a final circuit or diagram, but a reproducible derivation that records the transformation steps, the chosen cost model, and the resulting correctness claim.

The conceptual hierarchy in this section can therefore be summarized as follows. Reversible computation forms the exact information-preserving base layer; linear and affine $\mathbb{F}_2$ structure captures a tractable algebraic subhierarchy; Toffoli/Fredkin and Bennett cleanup provide universal reversible mechanisms with explicit resource discipline; and quantum computation generalizes the picture to unitary and channel semantics, where Clifford+ T and ZX-calculus supply the corresponding universal and diagrammatic frameworks. The unifying principle is that closure is always relative to a chosen notion of resource, error, and compositional equivalence.

3.3.6 13. Cryptographic Constructions & Symmetry Constraints

This section studies how symmetry constraints shape cryptographic design and how they can be used, carefully, in cryptanalysis. The central theme is that many security-relevant properties are not merely algebraic accidents of a particular implementation, but are tied to invariants under affine transformations, round composition, and group actions on the underlying state space.

A useful starting point is the avalanche behavior of a round function or full cipher. For a cipher family indexed by a round parameter r , one may summarize diffusion by an error profile $\varepsilon(r)$, interpreted as a measure of how strongly small input perturbations spread through the state after r rounds. In practice, this is often examined alongside differential and linear bias: differential behavior tracks how input differences propagate to output differences, while linear bias measures correlations between affine masks on input and output. These quantities are not independent of symmetry. If a design admits large automorphism groups or repeated structural patterns, then some difference or mask classes may be preserved or mapped into one another, producing regularities that

are visible in reduced-round experiments. In that sense, $\varepsilon(r)$ is not just a performance summary; it is also a probe for hidden invariants that survive partial composition.

S-boxes provide the most concentrated setting for symmetry analysis. Two S-boxes that are affine equivalent differ only by invertible affine changes of coordinates on the input and output, while CCZ equivalence is a broader relation that preserves several cryptographic invariants of interest. Under these equivalences, properties such as nonlinearity and differential uniformity are central. High nonlinearity helps suppress linear approximation, while low differential uniformity limits the number of solutions to difference equations and is therefore desirable for resistance to differential attacks. The symmetry viewpoint is useful here because it separates intrinsic cryptographic quality from superficial coordinate choices: an S-box may look different after a change of basis, yet remain in the same equivalence class with the same essential resistance profile. Conversely, two S-boxes that are not equivalent may still share some coarse metrics, so equivalence classes should be treated as a structural lens rather than a complete security certificate.

Round-based constructions such as SPN and Feistel networks make these issues more subtle. In an SPN, repeated application of substitution, permutation, and key mixing can amplify local S-box properties into global diffusion, but the composition also introduces invariants that may survive across rounds. In a Feistel structure, the split state and alternating update rule impose a different symmetry pattern, often yielding useful algebraic regularities for analysis. In both cases, one studies which quantities remain stable under round composition, which are merely permuted, and which are destroyed by the mixing layer. This is especially important when comparing full-round security claims with reduced-round experiments: a property that appears strong at one round count may be an artifact of a surviving symmetry rather than a robust indicator of long-term behavior. The practical lesson is that round count, key schedule, and state representation must be reported together, because each can change which invariants are visible.

Group actions provide a unifying language for these design and analysis questions. A cryptographic primitive can be acted on by affine coordinate changes, bit permutations, state relabelings, or more specialized equivalence relations, and the resulting orbits organize families of functionally related designs. From the design side, one often seeks to constrain symmetry so that the construction is not overly rigid; excessive symmetry can create exploitable structure. From the cryptanalysis side, one may exploit symmetry to reduce search spaces, classify equivalent instances, or identify invariant subspaces and fixed points. The same formalism also clarifies closure pitfalls: a class of constructions may be closed under a natural operation such as composition or conjugation, yet that closure can hide degenerate members whose apparent complexity does not translate into security. Closure under a transformation should therefore be read as a statement about membership in a family, not as a guarantee that every member is equally robust.

An important methodological caution is that symmetry-preserving transformations are not automatically security-preserving in the operational sense. Equivalence classes can preserve nonlinearity or differential uniformity while still changing implementation cost, side-channel exposure, or the interaction with a particular key schedule. Likewise, a family closed under a chosen group action may still contain weak representatives if the action is too coarse to distinguish them. For this reason, symmetry analysis should be paired with explicit metrics, round-reduced experiments, and implementation-aware checks rather than treated as a substitute for them. In particular, one should distinguish mathematical invariants from engineering invariants: the former may survive coordinate changes, while the latter depend on hardware, software, and leakage model.

The section therefore supports an artifact-driven workflow. An equivariance classifier can be used to identify which transformations preserve the relevant design class, and round-reduced experiments can probe how $\varepsilon(r)$, differential bias, and linear bias evolve as rounds are added. Such experiments are most informative when reported with the exact equivalence relation, the round

count, and the state representation used in the test harness. A useful artifact package would include the classifier’s decision rule, the canonicalization procedure used to compare instances, and a small suite of benchmark ciphers or S-box families for which the invariants are known. In the spirit of the book, the goal is not to claim security from symmetry alone, but to use symmetry as a disciplined lens for organizing constructions, pruning redundant cases, and exposing structural weaknesses.

Ethically, these tools should be used to improve robustness and to understand failure modes, not to facilitate misuse. The same methods that classify equivalent designs and accelerate cryptanalysis can also help designers avoid accidental structural weaknesses and can support reproducible evaluation of candidate primitives. When shared responsibly, symmetry-based analysis becomes a way to make cryptographic evaluation more transparent, more comparable across implementations, and less dependent on informal intuition.

3.3.7 14. Learning on $(\mathbb{B}^n)$ and Equivariant Architectures

This section studies learning problems on the Boolean cube $\mathbb{B}^n$ through the lens of symmetry, closure, and compositional structure. The guiding question is not merely how to fit a function $f : \mathbb{B}^n \rightarrow \mathbb{R}$ or $f : \mathbb{B}^n \rightarrow \mathbb{B}^m$, but how to do so in a way that respects the invariances of the task class and the algebraic constraints inherited from the surrounding system. In the present book, this means treating symmetry-aware learning as another instance of closure under admissible transformations: the hypothesis class should be stable under the relevant group action, and the learned representation should expose the same structural regularities that appear in circuits, clones, and compositional hierarchies.

A useful starting point is the generalization error under a symmetry class, denoted here by $\varepsilon(r)$, where r indexes the amount of symmetry or the radius of approximation under the chosen metric. Informally, $\varepsilon(r)$ measures how much predictive error remains when the learner is constrained to a family of models that is invariant, or approximately invariant, under a prescribed symmetry group. In the idealized case, the target function lies in a symmetry class $\mathcal{C}$ and the hypothesis class $\mathcal{H}$ is closed under the same action; then the effective sample complexity can improve because the learner need not rediscover equivalent configurations separately. In practice, however, symmetry constraints can also introduce bias if the true target only partially respects the assumed invariance. The role of $\varepsilon(r)$ is therefore to make explicit the tradeoff between structural restriction and approximation power.

Fourier analysis on $\mathbb{B}^n$ provides a canonical language for this tradeoff. Writing Boolean functions in the Walsh–Fourier basis decomposes them into coefficients indexed by subsets of coordinates, thereby separating low-order interactions from higher-order ones. This decomposition is especially useful when the target exhibits junta or sparsity structure. A junta depends on only a small number of coordinates, while a sparse Fourier spectrum concentrates most of its mass on a small number of coefficients. Both conditions are compatible with symmetry-aware learning: if a function is invariant under a permutation group, then its Fourier support is organized into orbits, and the effective number of distinct parameters may be much smaller than the ambient dimension suggests. Consequently, learning algorithms can exploit coordinate sparsity, spectral sparsity, or orbit sparsity, depending on which structural assumption is most faithful to the data.

The symmetry groups of interest are not limited to simple coordinate permutations. Group-equivariant networks generalize the familiar idea of convolutional weight sharing to arbitrary group actions, including permutation groups and wreath products. A network is equivariant when applying a group element to the input induces a corresponding, predictable transformation of the output. This property is valuable because it enforces consistency across symmetric instances and

reduces the burden on the optimizer. For example, if a task is invariant under relabeling of inputs, then a permutation-equivariant architecture can represent the same decision rule with fewer free parameters and with better inductive bias than an unconstrained multilayer perceptron. Wreath products arise naturally when the symmetry has a hierarchical or block-structured form, such as permutations within local neighborhoods together with permutations of the neighborhoods themselves. In such cases, the architecture should reflect the semidirect layering of the symmetry rather than flattening it into a single undifferentiated action.

From the perspective of closure, the central design question is whether the chosen constraints remain stable across layers. If each layer is equivariant with respect to a group action, then the composition of layers is equivariant as well, provided the intermediate nonlinearities and pooling operations are compatible with the action. This closure across layers is not automatic: a pointwise nonlinearity may preserve equivariance for some actions but not for others, and a normalization step may inadvertently break identifiability by collapsing distinct symmetric states. The architecture must therefore be checked as a whole, not merely layer by layer in isolation. In a compositional setting, one asks whether the class of admissible layers forms a closed family under the relevant operations, and whether the resulting model class remains identifiable up to the intended symmetry. Identifiability here means that equivalent parameterizations should correspond exactly to the same function modulo the symmetry, rather than introducing spurious degrees of freedom that obscure interpretation or hinder optimization.

This closure perspective also clarifies the relationship between representation learning and circuit extraction. If a learned model is constrained to respect a symmetry class, then its internal structure may be distilled into a smaller circuit or symbolic artifact that preserves the same invariance profile. Such distillation is especially attractive when the learned model is used as an intermediate artifact in a larger verified pipeline. The resulting model zoo can be organized by symmetry type, approximation regime, and compression level, allowing one to compare architectures not only by accuracy but also by equivariance, parameter efficiency, and extractability. In this setting, the artifact is not merely a trained network but a family of related representations: a dense model, a symmetry-restricted model, and a distilled circuit approximation that can be checked against the original behavior on a chosen test suite.

Privacy considerations enter naturally once symmetry-aware learning is deployed on sensitive data. Symmetry can reduce the effective hypothesis space, which may improve generalization, but it can also make certain attributes easier to infer if the symmetry class aligns with latent structure in the data. Moreover, distillation and model compression can leak information if the extracted artifact preserves too much of the training signal. A careful treatment therefore requires distinguishing between invariance that is beneficial for robustness and invariance that inadvertently exposes protected correlations. In practical terms, one should evaluate whether the symmetry constraints amplify membership inference, attribute inference, or reconstruction risk, especially when the model zoo includes distilled circuits or other compact artifacts intended for downstream reuse. Privacy-aware design may require additional regularization, noise injection, or access controls on the exported representations.

The overall lesson is that learning on $\mathbb{B}^n$ is most effective when the hypothesis class is aligned with the algebraic and symmetry structure of the task. Fourier methods reveal which interactions matter; junta and sparsity assumptions identify low-complexity targets; equivariant architectures encode invariance directly; and closure across layers ensures that the intended symmetry survives composition. At the same time, identifiability and privacy must be treated as first-class constraints, not afterthoughts. In the language of this book, a successful learning architecture is one whose symmetry constraints are compositional, whose representations are interpretable up to the intended equivalence, and whose exported artifacts remain faithful to both performance and governance

requirements.

3.3.8 15. Algorithms for Closure & Symmetry Analysis

Presents algorithmic methods for closure and symmetry analysis in Boolean and circuit settings, with an emphasis on executable decision procedures, canonical forms, and empirical validation. The central theme is that structural questions about clones, equivalence classes, and symmetries can often be reduced to search, constraint solving, or graph-theoretic normalization, but that each reduction comes with a measurable trade-off between soundness, precision, and runtime.

A useful way to summarize an implementation is by an error profile $\varepsilon(r)$, where r denotes a resource budget such as time, memory, or solver depth. In this setting, $\varepsilon(r)$ records the gap between the ideal mathematical decision procedure and the realized algorithm under bounded resources. For example, a SAT-based membership test may be sound but incomplete under a timeout, while a BDD-based procedure may be exact on some families and intractable on others. The practical question is therefore not only whether an algorithm is correct in principle, but how its soundness and precision behave as resources vary, and how often the implementation returns a definitive answer versus an inconclusive one. Benchmarks should report these rates explicitly, together with the fraction of instances solved by SAT, SMT, or symbolic reduction.

For clone membership, the basic problem is to determine whether a target Boolean function or circuit belongs to a specified clone generated by a finite basis. A common strategy is to encode the membership condition as a satisfiability problem: the unknown witness is a composition tree, a term over generators, or a structural assignment to internal nodes, and the constraints enforce semantic agreement with the target. When the clone is represented by a decision diagram or a normal form, the same question can often be phrased as a reachability or inclusion test. In practice, a hybrid SAT/BDD pipeline is effective: BDDs can simplify or prune the search space, while SAT handles combinatorial branching and generator selection. This is especially useful when the clone is given implicitly by a small generating set and the target object is large.

Generator discovery is the dual task of inferring a compact basis from observed behavior or from a family of derived operations. Here the algorithmic goal is to identify a minimal or near-minimal set of generators whose closure reproduces the observed clone fragment. Heuristics typically combine closure tests, redundancy elimination, and greedy basis refinement. A candidate generator is retained if it expands the closure in a way that cannot be simulated by the current basis within the chosen resource bound. Because minimal generating sets need not be unique, the output is often best understood as a canonical or stable basis relative to a fixed ordering and scoring rule rather than as an absolute mathematical invariant.

Canonicalization is the next major ingredient. For Boolean functions and circuits, graph isomorphism methods provide a general route to canonical forms by translating the object into a labeled graph whose automorphisms correspond to semantic symmetries. NPN canonicalization is a specialized and highly effective variant for Boolean functions, where negations of inputs and outputs and permutations of inputs are factored out. The canonical representative is chosen by exploring the orbit under these transformations and selecting the lexicographically least or otherwise preferred encoding. Heuristics matter here: variable ordering, partition refinement, and local invariants can dramatically reduce the search space before any expensive isomorphism test is invoked. In a circuit setting, canonicalization may also exploit gate types, fanout structure, and depth profiles to stabilize the search.

Equivalence checking asks whether two functions, circuits, or specifications denote the same behavior, possibly modulo a symmetry group. The standard reduction is to compare canonical forms, but direct equivalence procedures are often preferable when a full canonicalization is too

expensive. SAT-based equivalence checking introduces a miter circuit whose satisfiability witnesses a counterexample; UNSAT certifies equivalence. For richer theories or mixed Boolean-arithmetic fragments, SMT can extend the same pattern. The algorithmic design problem is to choose the right abstraction level: too coarse, and false positives proliferate; too fine, and the solver becomes impractical. Parameterized complexity provides a useful lens here, since many equivalence problems become tractable when measured by width, treewidth, generator count, symmetry rank, or another structural parameter. The resulting analysis clarifies which instances are expected to scale and which are inherently hard.

The complexity landscape is therefore multi-dimensional. Exact clone membership and equivalence checking are often computationally difficult in the worst case, but practical performance depends strongly on structural regularity. Parameterized algorithms can isolate tractable regimes, while heuristics and canonical forms provide fast filters for the common case. A typical workflow is to apply cheap invariants first, then canonicalization or symmetry reduction, and finally a solver-backed certificate check. This layered approach yields a robust pipeline: inexpensive tests eliminate easy negatives, normalization collapses equivalent instances, and the solver handles the residual core. The trade-off is that each layer introduces its own approximation profile, which should be measured rather than assumed.

An implementation-oriented section of the toolkit should therefore expose the following artifacts: a command-line interface for membership, canonicalization, and equivalence queries; a benchmark suite spanning random, structured, and adversarial instances; and logging that records solver choice, timeout behavior, canonical form statistics, and witness extraction. Such a CLI toolkit makes the algorithms reproducible and comparable across back ends. It also supports regression testing, since changes in heuristics can be evaluated against a fixed corpus of instances and a fixed reporting protocol.

In summary, the algorithmic study of closure and symmetry analysis is not merely about deciding membership or equivalence in the abstract. It is about building a reliable pipeline from algebraic specification to executable decision procedure, with explicit attention to $\varepsilon(r)$, solver success rates, canonicalization quality, and the structural parameters that govern complexity. The practical objective is a family of methods that are mathematically principled, computationally transparent, and suitable for integration into a reusable analysis toolkit.

3.3.9 16. Case Studies & Build Logs

This concluding section collects the book’s artifact-oriented case studies and build logs. Its purpose is not to introduce new theory, but to show how the preceding notions of closure, symmetry, modularity, and multi-resolution analysis behave in practice when one moves from abstract specifications to executable designs, optimization passes, and failure analysis.

A recurring theme is the use of multi-resolution dashboards, denoted here by $\varepsilon(r)$, to track a design across several levels of abstraction: function, gate, timing, and device. At the function level, one asks whether the intended Boolean behavior is preserved under rewriting, factoring, or decomposition. At the gate level, the concern shifts to structural cost, fan-in constraints, and local equivalence under identities. At the timing level, the same artifact is evaluated for critical path length, slack, and the stability of closure under retiming or pipelining. At the device level, the question becomes whether the implementation remains realizable under the physical constraints of the target technology, including area, power, and routing pressure. The point of $\varepsilon(r)$ is that a single scalar or pass/fail verdict is usually insufficient; closure must be monitored as a family of error or deviation measures indexed by resolution.

The first build log traces a progression from a NAND-only basis to a 4-bit ALU and then to

a small microcontroller-style datapath. The NAND stage serves as the minimal functional seed: from a single universal gate, one synthesizes the standard Boolean primitives and verifies that the resulting library is closed under the intended composition rules. The next stage lifts this basis into a 4-bit arithmetic logic unit, where the design problem is no longer merely functional completeness but compositional regularity. Here, the relevant questions include whether adders, multiplexers, comparators, and control logic can be assembled from a shared set of subcircuits without violating interface contracts, and whether the resulting hierarchy admits clean reuse of verified components. The final stage extends the same methodology to a microcontroller-like system, where the build log emphasizes integration rather than isolated correctness: instruction decode, register transfer, control flow, and memory-facing interfaces must all remain consistent under optimization and refinement.

Across these case studies, optimization is treated as a sequence of semantics-preserving transformations rather than as an ad hoc search for smaller netlists. One important family of transformations is compression by symmetry, especially LUT packing and factoring. If a circuit contains repeated or permuted substructures, then symmetry-aware factoring can identify common subexpressions, reduce duplication, and pack equivalent logic into fewer lookup tables or fewer structural blocks. In practice, this means that the build log records not only the final size of the artifact but also the symmetry classes exploited during optimization, the factoring choices made at each step, and the extent to which the transformed design remains equivalent to the source specification. The relevant metric is not just gate count, but the tradeoff between structural compression and the preservation of closure under the chosen equivalence relation.

A useful way to read these logs is as a sequence of checkpoints. At each checkpoint, the artifact is compared against the current specification at the appropriate resolution, and the result is recorded together with the transformation history. For example, a function-level equivalence certificate may be paired with a gate-level resource report and a timing report that exposes a newly critical path. This layered record makes it possible to distinguish genuine improvement from merely shifted cost: a rewrite that reduces area but destroys timing closure is not a success in the operational sense, even if it is algebraically valid. Likewise, a retiming pass that preserves behavior but obscures the structure needed for later verification may be acceptable only if the build protocol explicitly allows that loss of transparency.

The post-mortem component of the section records closure failures as counterexamples. These failures are valuable because they identify where a proposed hierarchy, rewrite system, or optimization pipeline does not actually close under composition, refinement, or implementation constraints. A closure failure may arise when a local rewrite is valid in isolation but breaks a global invariant; when a factoring step introduces an interface mismatch; when a timing optimization preserves functionality but violates the target clock budget; or when a symmetry reduction collapses distinctions that are operationally significant. Each counterexample is therefore documented with enough context to reproduce the failure, diagnose the violated assumption, and classify the failure mode relative to the book’s earlier closure notions.

In the most informative failures, the counterexample is not merely a bug report but a boundary marker for the theory. It shows where a closure claim must be weakened, where an equivalence relation must be refined, or where an optimization pass must be constrained by additional contracts. For instance, a transformation that is sound at the Boolean level may fail to respect a device-level constraint such as fanout, placement, or routing congestion. Conversely, a design that is physically realizable may fail to admit a clean algebraic factorization because the chosen decomposition destroys a symmetry that the later stages depend on. The logs therefore serve as a bridge between theorem and engineering practice: they show how abstract closure properties are tested, qualified, and sometimes rejected in the presence of real implementation constraints.

The artifact discipline is central to these logs. Each case study is accompanied by a repository index, schema definitions, and toolchain specifications so that the build can be replayed and audited. The repository index identifies the source artifacts, generated outputs, and intermediate checkpoints. The schemas specify the expected structure of netlists, timing reports, equivalence certificates, and optimization traces. The toolchain specification records the synthesis, verification, and analysis tools used, together with version constraints and invocation parameters. This makes the section not merely descriptive but reproducible: the reader can inspect the same pipeline that produced the reported results, compare alternative runs, and extend the open problems list with new experiments.

The open problems at the end of the section are intentionally practical. They include questions about how to define robust multi-resolution error metrics that remain meaningful across abstraction layers; how to automate symmetry-aware compression without obscuring debugging information; how to characterize the boundary between successful closure and acceptable approximation in large designs; and how to standardize build logs so that optimization histories can be compared across toolchains and technologies. In this sense, the section closes the book by returning from theorem and construction to the engineering reality of incomplete closure, partial success, and reproducible failure.

Artifact index. The accompanying repository index, schemas, and toolchain specifications are intended as living documents. They should record the exact build inputs, the transformation sequence, the verification checkpoints, and the unresolved issues associated with each case study. Readers are encouraged to treat these artifacts as templates for their own closure experiments, extending them with additional resolutions, alternative optimization passes, and new counterexamples as the theory and tooling evolve.

Build-log template. A minimal record for each experiment should include: the source specification; the target resolution(s) r used in the dashboard $\varepsilon(r)$; the sequence of rewrites, decompositions, and optimizations; the equivalence relation used for validation; the timing and device constraints, if applicable; and the final status, including any unresolved discrepancies. When a run fails, the log should preserve the smallest counterexample known, the stage at which closure was lost, and the hypothesis that best explains the failure.

Reproducibility note. The section is designed to be replayable from the repository index and schema files alone. Any future extension should preserve this property by keeping generated artifacts, intermediate checkpoints, and toolchain versions synchronized with the narrative description of the build.

Suggested reading of the logs. Read the case studies as a ladder of resolutions: first the functional specification, then the structural realization, then the timing and device constraints, and finally the failure modes that reveal the limits of the chosen abstraction. The value of the logs lies not only in successful closure, but in the explicit recording of where closure stops being available.

4 Bibliography

References

- [1] Saunders Mac Lane. *Categories for the Working Mathematician*. 2nd ed., Springer, 1998.
- [2] Tom Leinster. *Basic Category Theory*. Cambridge University Press, 2014.
- [3] Samuel Eilenberg. *Automata, Languages, and Machines, Volume A*. Academic Press, 1974.
- [4] Stanley Burris and H. P. Sankappanavar. *A Course in Universal Algebra*. Springer, 1981.
- [5] Emil L. Post. The two-valued iterative systems of mathematical logic. *Annals of Mathematics Studies*, 5, Princeton University Press, 1941.
- [6] Martin Markl, Steve Shnider, and Jim Stasheff. *Operads in Algebra, Topology and Physics*. AMS, 2002.
- [7] Brendan Fong and David I. Spivak. *Seven Sketches in Compositionality: An Invitation to Applied Category Theory*. Cambridge University Press, 2019.
- [8] David I. Spivak. *Category Theory for the Sciences*. MIT Press, 2014.
- [9] André Joyal, Ross Street, and Dominic Verity. Traced monoidal categories. *Mathematical Proceedings of the Cambridge Philosophical Society*, 119(3):447–468, 1996.
- [10] Peter Selinger. A survey of graphical languages for monoidal categories. In *New Structures for Physics*, Lecture Notes in Physics 813, Springer, 2011.
- [11] Bob Coecke and Aleks Kissinger. *Picturing Quantum Processes: A First Course in Quantum Theory and Diagrammatic Reasoning*. Cambridge University Press, 2017.
- [12] Bob Coecke, Aleks Kissinger, and others. ZX-calculus and related diagrammatic methods for quantum computation. Representative survey literature on categorical quantum mechanics and diagrammatic reasoning.
- [13] Michael Huth and Mark Ryan. *Logic in Computer Science: Modelling and Reasoning about Systems*. Cambridge University Press, 2004.
- [14] Glynn Winskel. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press, 1993.
- [15] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
- [16] Oded Goldreich. *Computational Complexity: A Conceptual Perspective*. Cambridge University Press, 2008.
- [17] Kazuo Murota. *Mathematics of Convex and Discrete Optimization*. SIAM, 2009.
- [18] Reinhard Diestel. *Graph Theory*. 5th ed., Springer, 2017.
- [19] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. 10th anniversary ed., Cambridge University Press, 2010.

- [20] Douglas R. Stinson. *Cryptography: Theory and Practice*. 3rd ed., Chapman & Hall/CRC, 2005.
- [21] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [22] Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv:2104.13478*, 2021.
- [23] Donald E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. 3rd ed., Addison-Wesley, 1997.
- [24] Garrett Birkhoff. Lattice theory. *American Mathematical Society Colloquium Publications*, 1940.
- [25] Nicholas Pippenger. Theories of computation and the closure of composition. Representative foundational literature on circuit complexity and compositional models.

5 Back Matter

5.1 Appendices

Appendix A: Post’s lattice cheat sheet; completeness tables. A compact reference for the clone-theoretic classification of Boolean function classes, including the principal nodes and inclusions in Post’s lattice, standard closure properties, and completeness criteria used throughout the book. This appendix is intended as a quick lookup table for functional completeness arguments, separation results, and recurring examples. It is organized for consultation rather than exposition, and it should be read alongside the chapters on closure, clones, and Boolean function classes.

Appendix B: Operad/PROP identities; coherence diagrams. A reference sheet for the algebraic identities and coherence laws that recur in the main text, including operadic substitution, PROP composition, symmetry and naturality constraints, and diagrammatic equalities. The emphasis is on the identities needed to verify that composite constructions are well-typed and that alternative parenthesizations or wiring orders agree. This appendix collects the equations and commuting-diagram patterns that support routine coherence checks in the book’s compositional arguments.

Appendix C: ZX, traced monoidal, decorated cospan guides. A compact guide to the diagrammatic formalisms used in the book: ZX-calculus conventions, traced monoidal feedback notation, and decorated cospan interfaces for open systems. This appendix serves as a bridge between the abstract categorical language and the string-diagram or network representations used in examples and case studies. It is intended as a notation guide and a reminder of the standard translation patterns between algebraic and graphical presentations.

Appendix D: Metrics catalog for $\varepsilon(r)$: defs, units, calibration. A catalog of the error and robustness metrics used to quantify approximation quality across resolutions, including the definition of $\varepsilon(r)$, the associated units and normalization conventions, and the calibration procedures used when comparing artifacts at different scales. The intent is to make metric usage consistent across chapters and to support reproducible reporting of numerical results. Where multiple conventions are possible, this appendix records the book’s chosen convention and the conditions under which comparisons are meaningful.

These appendices are supplementary reference material. They are designed to be consulted alongside the main chapters rather than read sequentially, and they collect the compact statements, identities, and conventions that support the book’s closure, compositionality, and symmetry arguments.

Use and scope. The appendices are intentionally concise: they prioritize lookup value over exposition, and they summarize the conventions, identities, and calibration rules that recur across the book. Where a chapter develops a concept in detail, the corresponding appendix entry should be read as a reminder of notation, a checklist of hypotheses, or a compact statement of the relevant law.

Cross-reference policy. Appendix A supports the Boolean closure and completeness arguments in the foundations chapters; Appendix B supports the algebraic and coherence calculations used in operadic and monoidal constructions; Appendix C supports the diagrammatic reasoning used in feedback, reversible, and quantum-flavored sections; and Appendix D supports the quantitative comparisons used in robustness, evaluation, and reproducibility discussions. Together, these appendices provide the reference layer for the book’s formal and computational claims.

Reading note. If a result in the main text depends on a convention, identity, or metric normalization, the corresponding appendix should be treated as the authoritative local reference. In particular, the appendices are meant to stabilize notation across chapters and to reduce duplication of standard material in the main narrative.

Scope of the appendix set. Appendix A is the lookup table for Boolean function classes and completeness criteria; Appendix B is the identity sheet for operads, PROPs, and coherence; Appendix C is the notation guide for ZX-calculus, traced monoidal structure, and decorated cospans; and Appendix D is the metric reference for $\varepsilon(r)$, including units, normalization, and calibration. Together they form the book’s compact reference layer for formal statements, diagrammatic conventions, and quantitative comparisons.

Consultation guide. When using these appendices, prefer the narrowest entry that matches the claim at hand: use Appendix A for Boolean-function classification and completeness checks, Appendix B for algebraic identity verification, Appendix C for diagrammatic translations and feedback conventions, and Appendix D for scale-dependent error reporting. This keeps the main chapters uncluttered while preserving a single authoritative location for each recurring convention.

5.2 Glossary & Symbol Index

Closure A property of a class of objects or operations being stable under the relevant composition rules used throughout the book. In the main text, closure is developed in the context of clones, operads, and compositional hierarchies; see the foundational chapters on closure theory and modular composition for formal statements and proofs.

Clone A set of Boolean functions closed under composition and containing the projection maps. Clones provide the algebraic language for describing which digital operations can be built from a chosen basis; see the chapter on Post’s lattice for the classification results.

Compositional hierarchy A layered organization of systems in which higher-level components are assembled from lower-level ones by explicit composition rules. The book uses this term to connect algebraic structure, circuit design, and modular refinement.

- Contract** A specification of assumptions and guarantees for a module or component. Contracts are used to state interface behavior and to support refinement and verification in the modularity chapters.
- Digital symmetry** A transformation of a digital object that preserves the structure of interest, such as a Boolean function, circuit, or dataflow graph. Symmetries may include permutations of inputs or wires, dualities, and other structure-preserving relabelings.
- Equivariance** The property that a map commutes with a symmetry action. Equivariant constructions appear in the chapters on symmetry groups, learning on $\mathbb{B}^n$, and parallel dataflow.
- Feedback** A composition pattern in which outputs are routed back into inputs. The book treats feedback through sequential composition, traced categories, and stateful machines.
- Functional completeness** The ability of a set of Boolean primitives to generate every Boolean function under composition. The NAND-based case study and the discussion of Post’s lattice use this notion as a central benchmark.
- Hierarchy** A stratified organization of components, abstractions, or proofs in which each level is built from the previous one. In this book, hierarchy is always tied to explicit composition and closure conditions rather than informal layering.
- Interface** The boundary description of a component, including its inputs, outputs, and typing or arity constraints. Interfaces are the primary objects of comparison in the modular and contract-based chapters.
- Invariance** The property that a quantity, relation, or behavior remains unchanged under a specified transformation. Invariance is used throughout the book to compare circuits, functions, and metrics across symmetries.
- Module** A component with a declared interface that can be composed with other components. Modules are the basic units of hierarchical design in the book’s construction-oriented chapters.
- Operad** An algebraic structure for composing multi-input operations according to arity and substitution rules. Operads provide one of the book’s main formalisms for compositional digital structure.
- PROP** A symmetric monoidal category whose objects are natural numbers and whose morphisms represent operations with multiple inputs and outputs. PROPs are used to model circuits, wiring diagrams, and related compositional systems.
- Refinement** A relation between specifications or implementations in which one artifact satisfies a stronger or more concrete version of another. Refinement is used to connect abstract contracts to executable builds.
- Reversible computation** Computation in which the mapping between inputs and outputs is bijective, so that information is not discarded. The reversible and quantum-flavored chapters use this notion to connect digital symmetry with physical constraints.
- Symmetry group** The group of all structure-preserving transformations of a given object. For Boolean functions and circuits, this often includes permutations and input/output relabelings that preserve behavior.

Trace An operation that closes a feedback loop in a monoidal setting. Traces formalize sequential composition with feedback and are used to reason about stateful systems.

$\mathbb{B}^n$ The n -dimensional Boolean cube, i.e., the set of all length- n bit vectors. It is the ambient space for Boolean functions, symmetry actions, and learning models in the book.

Symbol index. The table below provides a bidirectional map from symbols to their primary locations in the book. Page numbers are intentionally left as placeholders until the final pagination pass; chapter and section references are the stable navigation keys.

Symbol	Meaning	Primary location
$\mathbb{B}^n$	Boolean n -cube	Chapter 14, <i>Learning on $\mathbb{B}^n$ and Equivalences</i>
$\circ$	Composition of operations or maps	Front matter: Notation & Conventions
Bool	The Boolean domain / Boolean algebra context	Chapter 3, <i>Closure & Clones: Post's Lattice</i>
Clone	Composition-closed set of Boolean operations	Chapter 3
NAND	Sheffer primitive used for functional completeness	Chapter 7, <i>Case Study—From NAND to Boolean</i>
XOR	Exclusive-or gate / parity operation	Chapters 7, 8, and 14
AND	Conjunction gate	Chapters 3 and 7
OR	Disjunction gate	Chapters 3 and 7
NOT	Negation gate	Chapters 3 and 7
$\otimes$	Monoidal tensor / parallel composition	Chapter 2, <i>Operads, PROPs, and Symmetry</i>
Tr	Trace / feedback operator	Chapter 5, <i>Sequential Composition, Traces, and the Trace Monad</i>
η	Unit / identity-like structure, context dependent	Chapters 2, 4, and 6
ϵ	Counit / evaluation-like structure, context dependent	Chapters 2, 4, and 6
f	Generic function or morphism	Throughout the book; see Notation
g	Generic function or morphism	Throughout the book; see Notation
$\sim$	Equivalence relation / symmetry equivalence, context dependent	Chapters 8 and 15
$\cong$	Isomorphism / structural equivalence	Chapters 2, 4, 12, and 15
$\vdash$	Derivability / proof judgment	Chapter 6, <i>Constructive Logic: Heyting Arithmetic</i>

Location key. Chapter references are given by title so that the index remains stable even if pagination changes. When a symbol appears in multiple chapters, the listed location indicates the first or most central discussion; later uses should be read in light of that primary definition.

Reading note. For formal definitions, proofs, and build artifacts, follow the cross-references in the main text rather than relying on this glossary alone. This section is intended as a navigational aid and a plain-language reminder of the book's recurring symbols and terms.

How to use this index. The glossary is organized for two-way lookup: readers can move from a term to its plain-language meaning, then from a symbol to the chapter where it is first defined or used centrally. When a symbol is reused across several chapters, the index points to the most relevant entry point rather than attempting to enumerate every occurrence.

Scope note. This section is intentionally descriptive rather than exhaustive. It records the recurring vocabulary needed to navigate the book's proofs, constructions, and build artifacts, but it does not replace the formal definitions given in the chapters themselves.

5.3 Artifacts & Reproducibility

This section records the concrete artifacts that make the book’s claims inspectable, rebuildable, and testable. The intent is not merely archival: each item below is part of the reproducibility protocol for the technical development in the main text.

- **Manifest.** A manifest enumerates the code and data used in the project together with stable hashes. It should identify the relevant source files, generated outputs, and any auxiliary datasets or tables needed to reproduce a result. Where possible, the manifest should also record version identifiers for external dependencies so that a build can be reconstructed from the recorded state.
- **Build scripts.** Build scripts encode the steps required to transform source material into the published artifacts. These scripts should be deterministic, machine-readable, and sufficient to regenerate the book’s outputs from the manifest alone, subject to the stated environment assumptions.
- **Containers.** Container definitions capture the execution environment used for builds and checks. By pinning toolchain versions and system dependencies, containers reduce ambiguity about compiler behavior, library availability, and platform-specific differences.
- **Harnesses.** Test harnesses specify the checks used to validate claims, examples, and derived outputs. A harness may include unit tests, property tests, regression tests, or proof-checking scripts, together with short descriptions of what each test is intended to establish.
- **CI configuration.** Continuous integration configuration records how the build and test harnesses are executed automatically. This includes the order of checks, the environment in which they run, and the conditions under which a build is considered successful.
- **Outputs.** Outputs are the generated artifacts produced by the build and verification pipeline. These may include compiled documents, tables, logs, figures, and other derived files that substantiate the claims made in the text.

For each artifact class, the reproducibility standard is the same: the artifact should be identifiable, versioned, and linked to the procedure that produced it. When a result depends on a nontrivial computation or transformation, the corresponding manifest entry, script, and output should be kept together so that the chain from source to result remains auditable.

In practice, the reproducibility package for this book consists of the following minimal set:

1. a manifest of code and data with hashes;
2. the build scripts used to generate the book and its derived materials;
3. container definitions or equivalent environment specifications;
4. harnesses describing tests and verification steps;
5. CI configuration for automated execution;
6. the resulting outputs and logs.

Together, these artifacts support the book’s claims-as-builds methodology: every substantive claim should be traceable to a reproducible procedure, and every procedure should leave behind an inspectable output.

To make the protocol operational, the repository should keep the manifest, scripts, containers, harnesses, CI configuration, and outputs in a single auditable chain. A minimal implementation can be organized as follows:

- a top-level manifest that lists all tracked source files, generated files, and external inputs with hashes;
- one or more build scripts that regenerate the book and any derived tables, figures, or logs;
- a container or equivalent environment specification that pins the toolchain;
- test harnesses that describe the checks performed on claims, examples, and derived results;
- CI configuration that runs the build and tests in a fixed order;
- archived outputs that record the exact products of the pipeline.

This arrangement is intentionally conservative. It does not require every artifact to be self-explanatory in isolation; rather, it requires that each artifact be linked to the others in a way that permits inspection, rerun, and comparison against the recorded hashes. In that sense, reproducibility is not a single file or script but a disciplined relation among source, environment, procedure, and output.

For operational clarity, the repository should treat the following as the canonical reproducibility bundle:

- **manifest** files that enumerate inputs, outputs, and hashes;
- **build** scripts that regenerate the published materials;
- **container** or environment specifications that pin the toolchain;
- **harness** files that describe tests, checks, and expected outcomes;
- **ci** configuration that automates the build-and-test sequence;
- **outputs** and logs that record the exact results of the pipeline.

The practical criterion is simple: if a reader can start from the manifest, follow the recorded procedure, and recover the archived outputs under the stated environment, then the artifact set is sufficiently reproducible for the purposes of this book.

At a minimum, the reproducibility bundle should make the following items easy to locate and compare:

- the exact source snapshot used for the build;
- the hashes of generated files and external inputs;
- the scripts that transform source into output;
- the container image or environment specification used during execution;
- the test descriptions and CI job definitions;

- the archived outputs, logs, and any failure reports.

This section is therefore a checklist as much as a description. It names the artifacts that must exist, the relationships among them, and the evidence required to treat the book’s technical claims as reproducible builds rather than isolated assertions.

5.4 Errata & Change Log

Corrections. This section records substantive corrections, clarifications, and tracked revisions that affect the interpretation, notation, or results presented elsewhere in the book. When a later pass amends an earlier statement, the corrected form should be preferred, while the earlier form is retained here as part of the change history.

Version history. The change log provides a compact record of the book’s revision state, including the date or version identifier of each update, the scope of the modification, and a brief note on the affected material. This supports traceability across drafts and helps readers distinguish stable content from material that has been revised.

Limitations. The book is intentionally selective in scope. Some topics are treated at the level needed for the main narrative rather than as exhaustive surveys, and some constructions are presented with assumptions that should be checked against the intended application domain. Any such limitations are noted here when they materially affect interpretation or reuse.

Known issues and open problems. This log also tracks unresolved questions, gaps in coverage, and items deferred for future work. These may include proof obligations not yet discharged, implementation details awaiting validation, or extensions that remain open for further investigation.

The errata log is intended to remain a living record rather than a static appendix. As the manuscript evolves, entries should be updated to reflect corrected statements, newly identified limitations, and open problems that are relevant to readers who rely on the book as a technical reference. Where possible, each entry should identify the affected section or artifact and summarize the practical impact of the change.

For consistency with the rest of the book, future entries should be concise, versioned, and specific about scope. A useful entry format is: affected section or artifact, nature of the correction or limitation, version or date, and any downstream consequence for proofs, examples, or reproducibility artifacts. This keeps the log actionable for readers who need to determine whether a later revision changes a claim they depend on.